SFPNet: Sparse Feature Propagation Neural Network for Hyperspectral Image Classification

Xin Li, Xitong Xu, and Shengbo Chen

***Abstract*—Feature propagation models, represented by graph neural networks (GNNs) and Transformers, have achieved substantial progress in hyperspectral image (HSI) classification. However, existing approaches typically suffer from one of two limitations: patch-based strategies restrict the propagation range and hinder the modeling of cross-patch dependencies, while the superpixel-based propagation often struggles to produce fine-grained pixel representations. To address these issues, we propose a novel sparse feature propagation neural network (SFPNet) framework, termed SFPNet, which couples a pixel-level sparse feature propagation network (PFNet) with a superpixel-level sparse feature propagation network (SFNet). Specifically, PFNet constructs a structured sparse neighborhood by leveraging spectral and spatial correlations among pixels, and models fine-grained pixel dependencies via an efficient propagation operator, whereas SFNet captures long-range superpixel dependencies through a propagation mechanism with decayed neighborhood aggregation and state preservation. To accommodate different application scenarios, we further develop two PFNet variants with fixed and dynamic propagation weights, and couple each of them with SFNet to form two SFPNet architectures, namely, SFPNet-F and SFPNet-A. Few-shot classification experiments on six public benchmark datasets demonstrate that both SFPNet variants achieve consistently leading and robust performance across diverse distribution patterns and scale structures; compared with the best baseline on each dataset, SFPNet-F and SFPNet-A improve the overall accuracy (OA) by 1.98% and 2.25% on average, respectively. Moreover, modulewise ablations and spectral-domain analyses validate the strong fine-grained dependency modeling capability of PFNet-F/PFNet-A and the long-range modeling advantage of SFNet; operating under a unified sparse paradigm, the two modules exhibit clear complementarity, offering a generalizable design pathway for building efficient and robust feature propagation models for HSI classification. Our code is open-source on the public platform GitHub: https://github.com/LittleBlackBearLiXin/SFPNet**

***Index Terms*—Graph neural networks (GNNs), hyperspectral image (HSI) classification, Transformers.**

I. Introduction

HYPERSPECTRAL image (HSI) classification is a key technique with significant application value in high-precision sensing domains such as medical diagnosis [1], geological exploration [2], environmental monitoring [3], and military reconnaissance [4]. Unlike conventional RGB images that contain only three channels, hyperspectral images (HSIs) record tens to hundreds of continuous and narrow spectral bands at each pixel, spanning a broad electromagnetic range from the visible to the near infrared. This rich spectral information substantially enhances the ability to recognize and discriminate different ground objects or biological tissues. However, the high dimensionality of hyperspectral data, the complex nonlinear correlations across bands, and the pronounced heterogeneity and intricate geometric structures exhibited by samples of the same class in the spectral feature space pose severe challenges to classification. Traditional machine learning methods [5], [6], [7] are often hindered by the curse of dimensionality, feature redundancy, and limited generalization capability when dealing with HSI classification, which makes it difficult to achieve robust and efficient performance in real and complex scenarios.

With the rapid development of deep learning techniques [8], [9], [10], convolutional neural network (CNN)-based models have been widely introduced into HSI classification due to their mechanisms of local perception and weight sharing and have achieved notable success [11], [12], [13], [14]. Nevertheless, CNNs usually assume that the input data follow a regular grid structure, which leads to limitations in characterizing the irregular geometric patterns and complex contextual dependencies that are prevalent in hyperspectral data. In this article, feature propagation models dominated by graph neural networks (GNNs) [15], [16], [17] and Transformers [18] have attracted increasing attention due to their strong capability to model complex dependencies. For simplicity, following the systematic analysis by Joshi et al. [19] on the intrinsic connections between Transformers and GNNs, this article adopts the weighting scheme of feature propagation as the theoretical basis to distinguish the two model families. Specifically, feature propagation with fixed weights corresponds to the GNN paradigm, whereas feature propagation with dynamic weights realized by attention mechanisms corresponds to the encoder paradigm of Transformers.

In recent years, to address the complex spectral and spatial geometric features of HSI, a large number of modeling methods based on GNNs and Transformers have been proposed [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], [32], [33], [34], [35], [36], [37], [38], [39], [40], [41], and their classification performance has surpassed that of the mainstream CNN models. To systematically review representative

Received 10 August 2025; revised 10 January 2026 and 27 June 2026; accepted 31 July 2026. Date of publication 6 August 2026; date of current version 13 August 2026. This work was supported by the National Key Research and Development Program of China under Grant 2020YFA0714103. *(Corresponding author: Xitong Xu.)*

The authors are with the College of Geo-Exploration Science and Technology, Jilin University, Changchun 130026, China (e-mail: lixin17808627537@163.com; xuxitong@jlu.edu.cn).

Digital Object Identifier 10.1109/TGRS.2026.3720426

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

prior research, we classify these feature propagation models into the following two categories based on their basic units of feature propagation.

1) Pixel-level feature propagation models usually treat pixels as nodes in a graph or as feature tokens, and they directly construct geometric dependency relations at the pixel level to capture local or global spectral–spatial contextual features. Early studies built weighted graph structures based on spectral similarity or spatial proximity, and introduced GNNs with a transductive learning strategy to obtain global embedding representations for each pixel [20]. However, the adjacency matrices constructed by such methods are typically dense, and their computational complexity grows approximately quadratically with the number of nodes, severely limiting their applicability to large-scale HSI scenarios. To alleviate this bottleneck, methods in [21], [22], [23], and [24] partition HSIs into fixed-size image patches centered at foreground pixels, and adopt an inductive learning strategy to perform local modeling within each patch, which significantly improves computational efficiency. Building Subsequently, many works [21], [24], [25], [26], [27], [28], [29], [31], [32] further introduce Transformer architectures and use self attention within patches to densely learn finer grained dependencies among pixels, which further enhances the model expressive power. Although these patch-based strategies effectively reduce computational cost and achieve better performance than CNNs on multiple benchmark datasets, patch partitioning blocks feature propagation across patches, and it is therefore difficult to model pixel dependencies at the cross-pattern level.
2) Superpixel-level feature propagation models often apply superpixel segmentation algorithms [42] to divide an HSI into a set of compact superpixels and treat these superpixels as graph nodes or tokens, significantly reducing computational complexity by shrinking the effective problem radius. Early studies [33] constructed a local adjacency matrix based on spatial proximity among superpixels and used GNNs to learn local embedding representations for each superpixel. However, an increasing number of studies [43], [44], [45] indicate that the assumption that propagating features along local topological paths necessarily yields optimal node embeddings leads to several deficiencies in local GNN models, including heterophilic connections [43], excessive feature smoothing [44], and insufficient modeling of long-range dependencies [45]. To overcome these limitations, some researchers introduced attention mechanisms into the GNN framework to enhance the flexibility and globality of feature interactions. For example, the methods in [34], [35], and [36] adopt local attention to perform weighted aggregation over neighbor features and distinguish the importance of different neighbors, while the methods in [34], [35], [37], [38], [39], and [40] further use global attention to expand the scope of propagation to the global structure. Although these superpixel-level feature propagation models strike a certain balance between expressive power and scalability, pixel-level embeddings for downstream tasks are usually produced by assigning the superpixel-level features equally to all pixels contained in each superpixel. This practice ignores label heterogeneity within a superpixel and limits fine-grained classification capability in complex scenes. To address this issue, some mainstream approaches [34], [35], [36], [37], [38], [39], [40], [41] modularly integrate CNNs-based models [34], [46], [47] with superpixel-level feature propagation modules to generate fine grained pixel representations, but such designs may also inherit intrinsic limitations of CNNs, including uncertainty in kernel size selection, limited capability in distinguishing heterogeneous pixels, and difficulty in modeling long range dependencies.

In summary, pixel-level feature propagation models can capture fine intraclass variations, but they often incur high computational cost when modeling global dependencies, or their expressive power is constrained by patch size due to partitioning. In contrast, superpixel-level feature propagation models are more advantageous in computational efficiency and long-range dependency modeling; yet, they sacrifice pixel-level fine modeling because they implicitly assume homogeneity among pixels within each superpixel. Therefore, achieving unified modeling of local details and long-range dependencies while maintaining the computational efficiency has become a key challenge for HSI classification.

To tackle the above issues, we propose a sparse feature propagation neural network (SFPNet) framework for transductive HSI learning, termed SFPNet. The framework couples two core modules, namely, a pixel-level sparse feature propagation network (PFNet) module, PFNet, and a superpixel-level sparse feature propagation network (SFNet) network module, SFNet. In PFNet, we first construct a structured sparse neighborhood based on a joint metric of spectral similarity and spatial proximity. By combining patch partitioning with overlapping sliding windows, we significantly reduce the complexity of neighborhood construction while theoretically guaranteeing topological connectivity. Based on this structured neighborhood, we design two PFNet variants, PFNet-F with fixed propagation weights and PFNet-A with self-attention-based weights, in order to model fine pixel-level dependencies under different scenarios. In SFNet, since the superpixel graph is more compact in structure and more consistent in semantics than the original pixel graph, it can aggregate feature information from regions of the same class more effectively. We therefore construct neighborhoods using spatial proximity on the superpixel graph, and explicitly introduce decay aggregation and state preservation in each propagation step to enable long-range dependency modeling at the superpixel level. Through spectral domain analysis, we show that SFNet emphasizes large-scale graph structure while suppressing high-frequency noise, which allows it to effectively model long-range dependencies among superpixels with sufficiently many propagation layers, including the limiting case of infinitely many layers. By the parallel collaboration of two PFNet variants with SFNet, the two SFPNet variants can simultaneously achieve pixel-level fine-grained represen-

tation and superpixel-level long-range dependency modeling capabilities while adhering to the sparse computation paradigm throughout the entire process.

To systematically evaluate the effectiveness of the proposed method, we conduct extensive semi-supervised experiments on six HSI classification datasets and perform a comprehensive comparison with nine representative feature propagation models based on GNNs and Transformers. The results show that both SFPNet variants consistently outperform existing methods on all datasets. Compared with the state-of-the-art (SOTA) on each dataset, SFPNet-F achieves average improvements of 1.98%, 2.13%, and 2.70% in OA, AA, and Kappa, respectively. With the introduction of a dynamic attention mechanism, SFPNet-A performs even better, yielding corresponding average improvements of 2.25%, 2.48%, and 3.02%. Moreover, benefiting from the structured sparse neighborhood and the superpixel-level sparse propagation mechanism, SFPNet-F achieves a desirable tradeoff between computational scalability and representational capacity: on most datasets, it attains lower training and testing time than existing transductive feature propagation models while keeping GPU memory consumption within a controllable range. In addition, an analysis of scale-aware classification metrics indicates that both SFPNet variants maintain leading and well-balanced performance on small-scale complex patches as well as large-scale homogeneous regions, corroborating the synergistic advantage of the dual-path sparse propagation at pixel and superpixel levels for cross-scale modeling.

The main technical contributions of this article are summarized as follows.

1) We propose two pixel-level sparse feature propagation modules, PFNet-F and PFNet-A, based on fixed propagation weights and dynamic propagation weights, respectively. Both modules build structured sparse neighborhoods using joint spectral and spatial similarity, and perform efficient pixel-level feature propagation within these neighborhoods. This design enables the model to focus on critical contextual information while retaining computational scalability and the ability to capture fine dependencies.
2) We propose a flexible superpixel-level sparse feature propagation module, SFNet, which performs efficient multihop neighborhood aggregation via a feature propagation mechanism with decay aggregation and state preservation. The theoretical analysis shows that the propagation kernel of SFNet exhibits a stronger low-pass filtering property in the spectral domain, which enables effective long-range dependency modeling with sufficiently many propagation layers while mitigating the oversmoothing issue.
3) By coupling each PFNet variant with SFNet in parallel, we construct two SFPNet architectures. Both architectures achieve coordination between pixel-level fine-grained representation learning and superpixel-level long-range modeling within a unified sparse computation framework, and they significantly outperform several strong GNN and Transformer baselines for HSI classification on six standard datasets.

The remainder of this article is organized as follows. Section II reviews key properties of model paradigms related to our work. Section III presents the proposed SFPNet architecture and its key modules. Section IV describes the experimental settings and analyzes the results. Section V concludes this article and discusses potential extensions.

II. Related Works

A. Feature Propagation Models

As discussed in Section I, although CNNs have achieved initial success in HSI classification, their convolution and pooling operations rely on the assumption of a regular grid structure, which makes it difficult to directly model permutation-invariant representations among data with irregular geometric distributions. In contrast, feature propagation models represented by GNNs and Transformers have achieved remarkable results in a wide range of deep learning tasks due to their capability to capture latent geometric dependencies among data samples [48], [49], [50], [51]. Consider an input sequence $X \in \mathbb{R}^{N\times C}$, where N is the sequence length, and C is the feature dimension. Most feature propagation models can be formulated as follows:

$$X_i^{(l)} = f\left(\alpha_{ii}X_i^{(l-1)},\ \oplus\ \left\{\alpha_{ij}X_j^{(l-1)} : j \in \phi(i)\right\}\right) \tag{1}$$

where $X_i^{(0)} = X_i$ denotes the initial feature representation of pixel i. With l repeated iterations, the embedding function f and the permutation-invariant aggregation operator $\oplus$ are applied to learn the feature representation, or hidden state vector, $X_i^{(l)}$ at iteration l. The information is obtained from the self term and the propagation domain $\phi(i)$, both weighted by the scores $\alpha_{i:}$ from the previous iteration. Therefore, different designs of $\phi(\cdot)$, α, and $\oplus$ lead to different model architectures. For example, in a typical GNN [15], $\phi(\cdot)$ is a neighborhood structure defined by data priors, α is often implemented as a fixed mean function determined by the number of the self node and its neighbors, and $\oplus$ commonly uses summation. GraphSAGE [17] further extends $\oplus$ to include max pooling and mean aggregation. However, multistep iterations under the effect of Laplacian smoothing gradually pull the representations of adjacent nodes closer, and deep networks can easily suffer from the oversmoothing phenomenon [44]. Moreover, graph structure is central to GNNs, yet HSIs lack such naturally existing topology. They also exhibit challenges including high-dimensional nonlinear correlations among spectral features, strong redundancy across bands, and heterogeneity in both spatial and spectral domains. As a result, how to construct geometric priors suitable for HSI classification and how to design sparse feature propagation models remain key open problems.

In comparison, feature propagation in Transformers [18] is no longer constrained by a fixed structure $\phi(\cdot)$. Instead, Transformers learn global dependencies among sequences in a latent space via similarity functions, and they exhibit the following layerwise iterative property:

$$\alpha^{(l)} = \text{Softmax}\left(f_Q\left(X^{(l)}\right)\ f_K\left(X^{(l)}\right)^{\mathsf{T}}\right). \tag{2}$$

TABLE I

SUMMARY OF KEY PROPERTIES OF REPRESENTATIVE FEATURE PROPAGATION MODELS. HERE, N DENOTES THE NUMBER OF PIXELS ($N = H \times W$), k IS THE NUMBER OF SELECTED NEIGHBORS, B IS THE BATCH SIZE, AND p IS THE PATCH SIZE. M DENOTES THE NUMBER OF SUPERPIXELS. THE CORRESPONDING ADJACENCY MATRIX IS DENOTED BY A, AND THE NUMBER OF EDGES IS DENOTED BY $E(A)$

Idx	Method	Train	Unit	Dependency	Propagation	Complexity	Type
1	[20]	Transductive	Pixel	Data prior	Global	$O(N^2)$	GNN
2	[21]–[24]	Inductive	Pixel	Data prior	Local pixels within a patch	$O(Np^4)$	GNN
3	[21], [24]–[29], [31]	Inductive	Pixel	Latent space	Global pixels within a patch	$O(Np^4)$	Transformer
4	[32]	Inductive	Pixel	Latent space	Local pixels within a batch	$O(NB^2)$	Transformer
5	[33]	Transductive	Superpixel	Data prior	Local superpixels	$O(E(A))$	GNN
6	[34]–[36]	Transductive	Superpixel	Data prior and latent space	Local or global superpixels	$O(M^2 + N)$	Transformer
7	[37], [41]	Transductive	Superpixel	Data prior	Local superpixels	$O(E(A^k) + N)$	GNN
8	[37]–[39]	Transductive	Superpixel	Latent space	Local superpixels	$O(M^2 + N)$	Transformer
9	[40]	Transductive	Superpixel	Data prior and latent space	Global superpixels	$O(E(A) + M + N)$	Transformer
10	[31]	Inductive	Superpixel	Data prior	Local or global superpixels	$O(E(A) + N)$	GNN
11	[32]	Inductive	Superpixel	Latent space	Local superpixels	$O(M^2 + N)$	Transformer
12	**PFNet-F/A**	Transductive	Pixel	Data prior/Data prior and latent space	Local pixels	$O(Nk)$	GNN/Transformer
13	**SFNet**	Transductive	Superpixel	Data prior	Local superpixels	$O(E(A) + N)$	GNN

Here, f_Q and f_K are the embedding functions for queries and keys, respectively. We use single-head attention for illustration, and multihead attention is a natural extension. With a summation form of $\oplus$, Transformers can model the influence of different sequences more precisely as a weighted average of the self-feature and the features from other sequences. However, explicitly computing $\alpha^{(l)}$ leads to quadratic complexity, and the computational cost grows rapidly with the sequence length, which limits scalability on long-sequence data. To mitigate this bottleneck, some studies [52], [53], [54] replace the Softmax operator with linear kernel functions and use the associativity of matrix multiplication to realize approximate global feature propagation. Nevertheless, extensive empirical evidence shows that such linear attention methods often fail to preserve the focusing ability of Softmax [55], [56], [57], and their expressive power can even be inferior to that of several classical GNN models [15], [16], [17]. Therefore, in this work, we introduce a sparse attention mechanism tailored for HSI classification, aiming to achieve a desirable balance between expressive power and scalability.

B. Feature Propagation Models for HSI Classification

As shown in Table I, existing methods exhibit distinct focuses across training paradigms (direct inference/inductive learning), propagation units (pixels/superpixels), dependency modeling approaches (data priors/latent spaces/combined), propagation scope (local/global), and computational complexity. In contrast, the proposed PFNet-F/PFNet-A and SFNet correspond to pixel-level and superpixel-level sparse propagation modules in the table, respectively. They complement each other in dependency modeling and complexity control, forming the foundation for SFPNet-F/A to balance expressive power and scalability. To facilitate a clearer discussion of related work, we will now review feature propagation models from two categories: pixel-level and superpixel-level.

1) Pixel-Level Feature Propagation Models: In pixel-level feature propagation models, each pixel is treated as an independent feature unit. Beyond the key characteristics listed in Idx 1–4 of Table I, these methods have made substantial progress in technical design. For example, Yang et al. [21] and Huang et al. [24] constructed sparse local neighborhoods by computing the Euclidean distances between the center pixel and its neighboring pixels, which effectively suppresses noise propagation within a patch. Methods in [31] and [32] adopt a batch alignment strategy to fuse superpixel-level feature representations, thereby enhancing the modeling of spatial continuity. In [23], a dedicated loss function is designed to strengthen feature dependencies between the center pixel and similar neighbors. Works in [21], [24], and [25] improve patch partitioning strategies to better preserve local spectral and spatial characteristics. In [32], the center pixel is directly used as a feature token, which further improves computational efficiency. For feature preprocessing, Guo et al. [22], Feng et al. [26], and Yang et al. [32] employed 2-D CNNs, while Sun et al. [28] and Jia et al. [29] used 3-D CNNs to generate token representations that are more suitable for attention mechanisms.

Although these pixel-level feature propagation models provide valuable insights for HSI classification, patch-based strategies disrupt the spatial continuity of HSIs and limit the range of feature propagation. In contrast, the proposed PFNet-F/PFNet-A is the only pixel-level method that supports transductive learning while remaining computationally scalable. Moreover, PFNet-F/PFNet-A does not rely on a fixed patch size or dense self-attention. Instead, it adaptively constructs a structured sparse neighborhood for each pixel based on joint spectral and spatial correlations. The proposed subgraph partitioning and sliding window technique further guarantees both sparsity and connectivity of feature propagation.

2) Superpixel-Level Feature Propagation Models: In superpixel-level feature propagation models, the input $X \in \mathbb{R}^{N \times C}$, with $N = H \times W$, is first processed by a superpixel segmentation algorithm to obtain a superpixel set $\mathcal{S} = \{\mathcal{S}_i\}_{i=1}^{M}$. A pixel-to-superpixel association matrix $Q \in \mathbb{R}^{N \times M}$ is then used to convert between pixel features and superpixel features. The association matrix Q is defined as follows:

$$Q_{i,j} = \begin{cases} 1, & \text{if } X_i \in \mathcal{S}_j \\ 0, & \text{otherwise.} \end{cases} \tag{3}$$

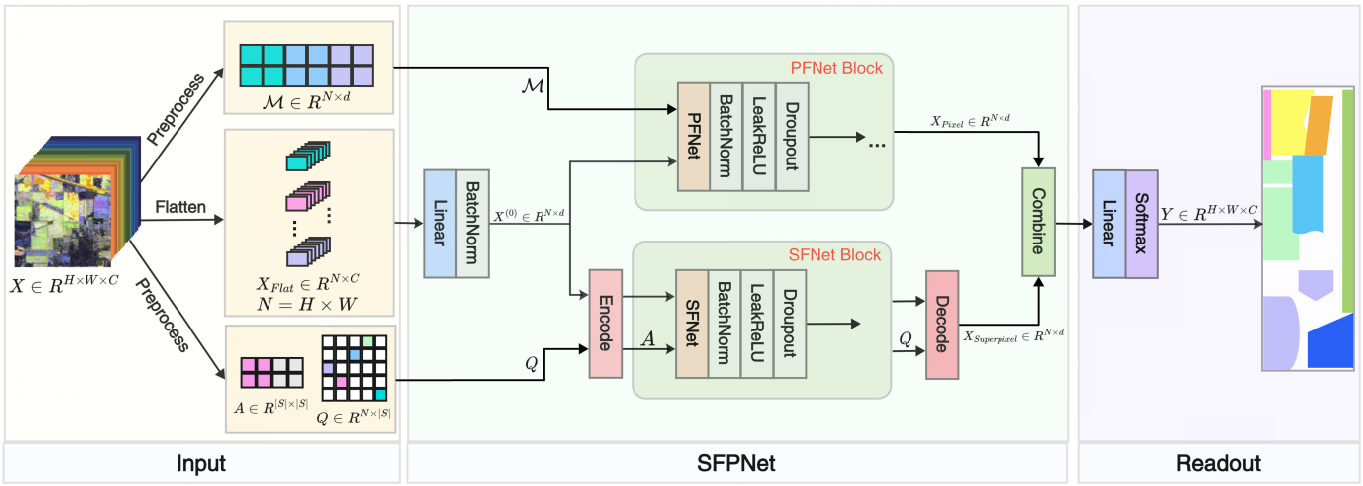


Fig. 1. Simplified architecture diagram of the proposed SFPNet is shown. This architecture employs a multilayer SFNet to handle superpixel-level long-range dependency modeling, combined with a 1–2 layer PFNet (configurable to optionally incorporate attention mechanisms) to enhance pixel-level fine-grained representation. This approach maintains computational sparsity while balancing the modeling of local details and long-range dependencies.

The conversion from pixel features to superpixel features is defined by an encoder as follows:

$$H^{(0)} = \text{Encode}\,(Q, X) = \text{Norm}\,(Q)^{\mathsf{T}}\, X \tag{4}$$

where Norm(Q) denotes the column-normalized version of Q. A feature propagation model is then applied on the superpixel feature graph to learn the final superpixel representations. As shown in Idx 5–11 of Table I, these models have achieved substantial progress at both local and global aggregation levels. In addition, Wan et al. [33] and Liu et al. [35] fused feature propagation models across multiple superpixel graphs constructed under different superpixel segmentation scales. Method [40] adopts NodeFormer [52] to jointly model local and global dependencies with linear complexity. Works in [37] and [41] develop a mixed propagation mechanism for multiscale neighborhood features through power operations of the adjacency matrix. After learning superpixel-level representations, a decoder maps them back to pixel-level features

$$X = \text{Decode}\left(Q, H^{(l)}\right) = QH^{(l)}. \tag{5}$$

To recover pixel-level fine-grained representations, many studies couple superpixel-level feature propagation models with CNN architectures. Methods in [35], [37], [39], [40], and [41] perform systematic optimization over the depth and width of grouped convolutions and over kernel scales. Methods in [36] and [38] introduce CNN modules based on channel attention and cross-attention to extract higher level pixel representations.

In comparison, the proposed SFNet constructs a neighborhood aggregation mechanism on the superpixel graph with exponential decay and state preservation. From a theoretical perspective, we first prove the convergence of the SFNet propagation kernel, which avoids the phenomenon in conventional deep GNN stacking where representations converge to a limiting distribution that is independent of the starting node. Further spectral-domain analysis shows that the Laplacian eigenvalue transformation induced by SFNet relatively amplifies low eigenvalues that describe large-scale structure and suppresses high eigenvalues associated with local noise and fine-grained perturbations. Therefore, we deliberately use SFNet, which operates on a smaller input scale, to perform more propagation steps and efficiently capture superpixel-level long-range dependencies. We then use PFNet-F/PFNet-A, which operates on a larger input scale, to perform fewer propagation steps and capture fine dependencies among pixels. By simply coupling SFNet with pixel-level PFNet-F/PFNet-A in parallel, we construct the SFPNet-F/A architecture that provides both fine-grained representation capability and long-range modeling capability.

III. Methodology

A. SFPNet Framework

The overall architecture of the proposed SFPNet is illustrated in Fig. 1. Given an HSI, denoted as $X \in \mathbb{R}^{H\times W\times C}$, where H, W, and C represent the image height, width, and spectral dimension, respectively. In the transductive learning setting for HSIs, each foreground location corresponds to a spectral signal and its associated class label. The goal of SFPNet is to learn a mapping function by minimizing the cross-entropy loss between the predictions and a subset of available class labels, so that accurate class predictions can be produced for every foreground pixel. In what follows, we introduce the overall framework of SFPNet from three aspects.

1) Input: The inputs required by SFPNet include a pixel feature matrix $X \in \mathbb{R}^{N\times C}$ with $N = HW$, a pixel-level prior topology $M \in \mathbb{R}^{N\times N}$, a pixel-to-superpixel association matrix $Q \in \mathbb{R}^{N\times M}$, and a superpixel-level prior topology $A \in \mathbb{R}^{M\times M}$. The matrix $\mathcal{M}$ required by the PFNet branch will be detailed in Section III-B1. For the matrices Q and A required by the SFNet branch, we reuse the method of Liu et al. [34], which has been widely adopted in this field, and we therefore omit further details.

2) SFPNet: SFPNet does not rely on dimensionality reduction algorithms. Instead, it obtains low-dimensional pixel embeddings through a linear transformation followed by batch normalization

$$X^{(0)} = \mathrm{BN}(f(X)), \quad X^{(0)} \in \mathbb{R}^{N\times d},\ d \ll C \tag{6}$$

where $f(\cdot)$ denotes a linear transformation. The embeddings $X^{(0)}$ are then fed into two branches, PFNet and SFNet, for feature extraction. PFNet outputs pixel-level fine-grained representations $X^{(l)}$ layer by layer. SFNet first obtains initial superpixel features $H^{(0)}$ via the encoder [see (4)] defined on Q, then produces region-level representations $H^{(r)}$ after r SFNet layers, and finally projects them back to the pixel domain via the corresponding decoder [see (5)] to obtain $X^{(r)}$.

SFPNet then fuses the features produced by the PFNet and SFNet branches to achieve effective integration of fine-grained features at the pixel-level and superpixel-level region features. Let $\mathcal{P}(\cdot)$ be one PFNet layer and $\mathcal{S}(\cdot)$ be one SFNet layer. The iterative process of SFPNet can be written as follows:

$$\begin{aligned} Z &= \mathcal{P}\left(X^{(l-1)}, M\right) + Decode\left(Q, \mathcal{S}\left(H^{(r-1)}, A\right)\right) \\ X^{(l)} &= \mathcal{P}\left(X^{(l-1)}, M\right) \\ H^{(r)} &= \mathcal{S}\left(H^{(r-1)}, A\right), H^{(0)} = Encode\left(Q, X^{(0)}\right) \end{aligned} \tag{7}$$

where the specific definitions of $\mathcal{P}(\cdot)$ and $\mathcal{S}(\cdot)$ will be further described in Sections III-B2 and III-C, respectively.

3) Readout: Given the fused pixel features $Z \in \mathbb{R}^{N\times d}$, we apply a Softmax classifier with a linear transformation to predict the class label for each pixel using the maximum probability

$$Y = \arg\max\left(\mathrm{Softmax}(f(Z))\right) \tag{8}$$

where $f(Z) = ZW + b$, with $W \in \mathbb{R}^{d\times\mathcal{C}}$ and $b \in \mathbb{R}^{\mathcal{C}}$. Here, $\mathcal{C}$ denotes the total number of classes.

B. Pixel-Level Sparse Feature Propagation Network

In this section, to address the limited propagation domain induced by existing patch-based feature propagation models, we propose a PFNet, termed PFNet. Before presenting the detailed architecture of PFNet, we first introduce a novel neighborhood construction algorithm in Section III-B1. This algorithm exploits the spectral and spatial priors embedded in HSIs to build a structured sparse neighborhood. Building on this neighborhood, we further design an efficient pixel-level feature propagation mechanism to model fine-grained semantic dependencies among key pixels.

1) Structured Sparse Neighborhood: Consider an HSI cube on a regular grid, denoted by $X \in \mathbb{R}^{N\times C}$ with $N = HW$. Each pixel i corresponds to a feature vector $X_i \in \mathbb{R}^C$, $i \in \{0,\ldots,N-1\}$, and a grid coordinate $C_i = (\lfloor i/W \rfloor,\ i \bmod W)$. An ideal structured neighborhood should constrain feature propagation to occur among semantically similar pixels, so as to obtain highly discriminative intraclass feature representations. Existing methods often adopt spatial k nearest neighbors as a surrogate for the ideal neighborhood. However, on a grid, the number of candidate neighbors that are equidistant to the center pixel increases rapidly as the relative distance grows, which makes neighbor selection ambiguous and difficult.

To address this issue, we propose a joint spectral and spatial similarity metric $\mathrm{Sim}(\cdot)$. This metric is nonnegative and bounded, and it follows the empirical prior that samples from the same class tend to be spectrally similar and spatially clustered. It can be defined as follows:

$$\begin{aligned} \mathrm{Sim}\left(X_i, X_j\right) &= \max\left(0,\ \hat{X}_i^{\mathsf{T}}\hat{X}_j\right) \cdot \exp\left(-\frac{d_{ij}}{2\gamma}\right) \\ \hat{X}_i &= \frac{X_i - \mu_i}{\sigma_i}, \quad \hat{X}_j = \frac{X_j - \mu_j}{\sigma_j} \\ d_{ij} &= \|C_i - C_j\|_2^2 \end{aligned} \tag{9}$$

where $\gamma > 0$ is the Gaussian kernel smoothing coefficient that controls the decay rate of spatial similarity. μ_i and μ_j are the means of pixel features X_i and X_j, and σ_i and σ_j are the corresponding standard deviations.

Based on this metric, we define the structured neighborhood of pixel i as follows:

$$\mathcal{M}(i) = \left\{ j \mid j \in \text{Top-}k\left(\mathrm{Sim}\left(X_i, X_q\right), q \in \epsilon\right)\right\} \tag{10}$$

where $\epsilon = \{0,\ldots,N-1\}$, Top-$k(\cdot)$ selects the k pixels with the highest similarity values under $\mathrm{Sim}(\cdot)$ from all pixels. Although (10) unifies spectral and spatial similarity modeling by combining a nonnegative Pearson-correlation term with a Gaussian distance kernel, the construction of $\mathcal{M}$ has a computational complexity as high as $\mathcal{O}(N^2 \log k)$. For high-resolution HSIs, this quadratic complexity severely limits practical usability and scalability.

To resolve this issue, we propose an optimization strategy that integrates blockwise processing and sliding windows, as shown in Algorithm 1. Specifically, motivated by the distance decay property of the Gaussian kernel and the need for a highly sparse topology, we first partition the HSI along height H and width W into b^2 equal-sized blocks. This avoids redundant computation for long-distance pixel pairs. Second, within each block, we preselect five nearest neighbors for each node according to the spatial Gaussian distance, in order to guarantee connectivity of the within-block topology. We then use (10) to select the remaining $k-5$ most similar neighbors so as to retain the representation capability of $\mathrm{Sim}(\cdot)$. Finally, to break block isolation and enlarge the sampling range, we traverse the image using a nonoverlapping sliding window with stride $(\lfloor H/2 \rfloor,\ \lfloor W/2 \rfloor)$ to construct the complete structured neighborhood. The pseudocode in Algorithm 1 provides an efficient implementation of this procedure.

It is worth noting that although we perform blockwise processing, the sliding window and neighborhood sampling design still provides the following theoretical guarantee.

Theorem 1: Let the input data cube on a regular grid be $X \in \mathbb{R}^{H\times W\times C}$, where $H, W \in \mathbb{Z}^+$ be the grid height and width, and $HW > 5$. Each pixel i corresponds to a feature vector $X_i \in \mathbb{R}^C$, $i \in \{0,\ldots,N-1\}$, and a grid coordinate $C_i = (\lfloor i/W \rfloor,\ i \bmod W)$. Let the structured neighborhood set $\mathcal{M} = \{\mathcal{M}(i)\}_{i=0}^{N-1}$ be constructed by Algorithm 1. If the neighbor number k satisfies $HW/b^2 \geq k \geq 5$, with $k, b \in \mathbb{Z}^+$, then the graph G defined by the structured neighborhood $\mathcal{M}$ is connected.

The proof is provided in Appendix A. According to Theorem 1, the propagation range of feature propagation models built on $\mathcal{M}$ is no longer restricted to local blocks. Instead, it

Algorithm 1 Sparse Structure Neighborhood Construction Algorithm

Input: $x \in \mathbb{R}^{h\times w\times c}$; b, k;
Output: m_index; ▷ Index Form of the Sparse Structure Neighborhood $\mathcal{M}$

Ineialize: m_index, sub_m_index, idx_top, $idx_toprest \leftarrow [\,], [\,], [\,], [\,]$;
$block_h \leftarrow h\,//\,b$; $block_w \leftarrow w\,//\,b$; $stride_h \leftarrow block_h\,//\,2$; $stride_w \leftarrow block_w\,//\,2$;
$coordinates \leftarrow connect(arange(h), arange(w))$;
function SELECT_NEIGHBORS(x, $coordinates$, k)
  $super \leftarrow sum(coordinates \odot coordinates, dim{=}1).reshape(-1, 1)$;
  $dis \leftarrow super.reshape(-1, 1) + super.reshape(1, -1) - 2 * dot(coordinates, coordinates^{\mathsf{T}})$;
  $sim \leftarrow exp(-dis/2)$; $idx_top.append(sim.topk(5, dim{=}-1).indices)$; $sub_m_index \leftarrow idx_top$; ▷ **Step3**
  if $k > 5$ **then**
    $x \leftarrow standardization(x)$; $sim \leftarrow \max(0, dot(x, x^{\mathsf{T}})) \odot sim$; $sim[sub_m_index] \leftarrow -INF$;
    $idx_toprest \leftarrow sim.topk(k-5, dim{=}-1).indices$; $sub_m_index.append(idx_toprest)$; ▷ **Step4**
  end if
  return sub_m_index
end function
for i in $range(0, h+1, stride_h)$ **do** ▷ **Step1**
  for j in $range(0, w+1, stride_w)$ **do** ▷ **Step2**
    $start_row \leftarrow i$; $end_row \leftarrow min(i + block_h, h)$; $start_col \leftarrow j$; $end_col \leftarrow min(j + block_w, w)$;
    $block_mask \leftarrow (coordinates[:,0] \geq start_row)$ & $(coordinates[:,0] < end_row)$; / & $(coordinates[:,1] \geq start_col)$ & $(coordinates[:,1] < end_col)$;
    $curr_index \leftarrow where(block_mask)[0]$;
    $m_index.append(SELECT_NEIGHBORS(x[curr_index], coordinates[curr_index], min(k, len(curr_index))))$
  end for
end for
$m_index.unique$;

can capture semantic information from farther spatial locations on the grid through the hierarchical stacking of model layers. In addition, Algorithm 1 significantly reduces the computational complexity to a controllable level of $\mathcal{O}(N^2 \log k/b^2)$. Meanwhile, the sparsity ratio of the adjacency matrix induced by $\mathcal{M}$ is at least $(N - 4k)/N$. When k is fixed, this sparsity ratio approaches the lower bound as the number of blocks b^2 increases. This optimization ensures strong applicability to HSIs at arbitrary scales and provides a solid basis for practical deployment.

2) Pixel-Level Sparse Feature Propagation Network: Based on the structured sparse neighborhood $\mathcal{M} = \{\mathcal{M}(i)\}_{i=0}^{N-1}$, constructed for each pixel node in the previous subsection, we now present the corresponding pixel-level sparse feature propagation network, termed PFNet. To accommodate different data scenarios, where $\mathcal{M}$ may exhibit homophily or heterophily, and to balance efficiency and expressive power, we design two feature propagation mechanisms according to the weighting scheme in feature propagation, namely, PFNet-F with fixed weights and PFNet-A with dynamic weights. The two variants share a unified propagation framework. Their layerwise propagation can be generally written as follows:

$$X^{(l)} = \mathcal{P}\left(X^{(l-1)}, M\right) = \left\lfloor X_0^{(l)}, X_1^{(l)}, \ldots, X_{N-1}^{(l)} \right\rfloor$$
$$X_i^{(l+1)} = f\left(X_i^{(l)}, \text{Add}\left\{m_{ij}^{(l)} X_j^{(l)} : j \in \mathcal{M}(i)\right\}\right) \tag{11}$$

where $X_i^{(l)}$ denotes the feature representation of pixel i at layer l, $\oplus$ denotes a permutation-invariant aggregation operator, and $f(\cdot)$ denotes a linear transformation.

a) PFNet-F: Similar to the conventional GCN [15], each PFNet-F layer propagates neighborhood features using predefined weights $m_{ij}^{(l)} = 1/(|\mathcal{M}(i)|\,|\mathcal{M}(j)|)^{1/2}$. Since the final pixel representation is explicitly determined by the features of pixels within its neighborhood, PFNet-F is suitable for homophilic scenarios, where the neighborhood structure $\mathcal{M}$ is of high confidence and most connected pixels belong to the same class.

b) PFNet-A: To further enhance the ability to capture fine-grained semantic dependencies among pixels, PFNet-A introduces dynamic attention weights as in the Transformer encoder. The propagation weight $m_{ij}^{(l)}$ is computed by a semantic correlation operator with Softmax in the latent space

$$m_{ij}^{(l)} = \text{Softmax}\left(f_Q\left(x_i^{(l-1)}\right) \cdot f_K\left(x_j^{(l-1)}\right),\ j \in \mathcal{M}(i)\right) \tag{12}$$

where $f_Q(\cdot)$ and $f_K(\cdot)$ are the linear projections for queries and keys, respectively. To leverage the symmetry of mutual connections among same-class pixels, we apply the same linear transformation to pixel features and set $f_Q(\cdot) = f_K(\cdot)$. In addition, since relative positional relations among pixels have already been considered during the construction of $\mathcal{M}$, PFNet-A does not require explicit positional encodings, which further simplifies the model. Owing to the dynamic nature of attention weights, PFNet-A can adaptively focus on the most discriminative contextual information within the neighborhood. It is therefore more suitable for data scenarios with complex connectivity and stronger heterophily.

c) Computational complexity: Since both PFNet variants propagate features along the predefined paths in $\mathcal{M}$, their main computational cost depends only on the number of

paths, which is Nk. Here, N is the number of pixels, k is the controllable neighbor number in Algorithm 1, and d is the feature dimension. Therefore, the complexity of a single PFNet layer is $\mathcal{O}(Nkd)$, which grows linearly with respect to k. It should be noted that the main cost of PFNet-F per layer consists of neighborhood feature aggregation and a single-linear transformation. In comparison, under the same propagation framework, PFNet-A additionally performs a linear projection, sparse attention weight computation, and Softmax normalization, which leads to a larger constant factor than PFNet-F.

C. SFNet

In the previous section, we constructed a sparse structured neighborhood at the pixel level and proposed PFNet-F/PFNet-A to efficiently model contextual dependencies among key pixels within local neighborhoods. Although PFNet-F/PFNet-A has linear complexity with respect to neighbor number k, the total input sequence length is $H \times W$. Therefore, simply stacking many PFNet-F/PFNet-A layers at the pixel level to capture long-range dependencies is inefficient. To address this issue, this section proposes a novel SFNet, termed SFNet. Its core idea is to introduce a propagation mechanism that combines attenuated aggregation and state preservation, which can both efficiently model long-range dependencies on superpixel maps and mitigate the problem of excessive smoothing during deep network propagation.

Given a superpixel graph $G = (H, A)$, where $H \in \mathbb{R}^{|S| \times d}$ is the node feature matrix, $|S|$ is the number of superpixels, and $A \in \mathbb{R}^{|S| \times |S|}$ is the adjacency matrix, the renormalized propagation matrix in the classical GCN framework is defined as $P = \tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}}$ with $\tilde{A} = A + I$, where $\tilde{D}$ is the degree matrix of $\tilde{A}$. Wu et al. [58] show that the addition of self-loops shrinks the eigenvalue range of the normalized Laplacian to $\lambda_i \in [0, 2)$, which improves the training stability. However, constrained by spectral graph theory, the rth-order polynomial filter $g(\lambda_i) = (1 - \lambda_i)^r$ converges, as the propagation step r increases, to a steady-state distribution that is independent of nodes. This property dilutes the distinctive local information of the nodes, leading to the oversmoothing phenomenon.

To mitigate this issue, we first introduce a decay factor $\beta \in (0, 1)$ to weight the aggregated neighborhood information at each step, which reflects the physical intuition that effective information naturally decays as the number of hops increases. Second, to prevent the loss of intrinsic node information during propagation, we reinject the initial node features H^0 after each feature update to achieve state preservation. Based on these designs, we define the $(r + 1)$th layer update of SFNet as follows:

$$H^{(r)} = \mathcal{S}\left(H^{(r-1)}, A\right) = \beta P H^{(r-1)} + \beta P f\left(H^0\right). \tag{13}$$

By iterative expansion, the above update rule can be equivalently written as follows:

$$H^{(r)} = (\beta P)^r f\left(H^0\right) + \sum_{i=1}^{r} (\beta P)^i f\left(H^0\right). \tag{14}$$

When the number of propagation steps is sufficiently large, with $r \to \infty$, the first term tend to 0, while the second term becomes a geometric series. Since $\beta \in (0, 1)$ and the spectral radius $\rho(\beta P) < 1$, this geometric series converges. This indicates that the SFNet limiting characteristic representation depends on the initial features $f(H^{(0)})$ and the limiting propagation matrix $\sum_{i=1}^{\infty} (\beta P)^i$, rather than being determined by the continuous iteration of intermediate representations. Unlike conventional deep GCN propagation, where node representations tend to gradually become similar, SFNet preserves an explicit dependence on the initial superpixel features through decayed propagation and state preservation, thereby mitigating information dilution and representation convergence in deep propagation. It should be noted that, when we set $\beta = (2 + \gamma)^{-1}$ with $\gamma \in \mathbb{R}^+$, we can further obtain the following theoretical guarantee.

Theorem 2: Let the normalized propagation matrix with self-loops be $P = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$, whose normalized Laplacian has eigenvalues $\lambda_i \in [0, 2)$. Let the SFNet limiting propagation matrix be $\tilde{P} = \sum_{i=1}^{\infty} (\beta P)^i$, where $\beta = (2 + \gamma)^{-1}$ and $\gamma \in \mathbb{R}^+$. Denote the eigenvalues of the Laplacian corresponding to $\tilde{P}$ as $\tilde{\lambda}_i$. Then, λ_i and $\tilde{\lambda}_i$ share the same set of eigenvectors. Moreover, when $\lambda_i \in [0, 1]$, we have $\lambda_i \leq \tilde{\lambda}_i$. When $\lambda_i \in (1, 2)$, we have $\lambda_i \geq \tilde{\lambda}_i$.

The proof is provided in Appendix A. Since low-frequency eigenvalues of the normalized Laplacian describe large-scale graph structure [59], and high-frequency eigenvalues correspond to local details and noise [58], the above relationship indicates that SFNet increases smaller eigenvalues and decreases larger eigenvalues. Therefore, SFNet strengthens the modeling of large-scale structures while suppressing high-frequency noise, which is beneficial for capturing long-range dependencies.

In addition, the decay factor $\beta = (2 + \gamma)^{-1}$ controls the decay rate of information during propagation. As γ increases, faster decay reduces the transmission of neighborhood information, which, in turn, reduces the high-frequency perturbations introduced by distant nodes. From a spectral perspective, increasing γ increases the eigenvalue transformation difference $|\tilde{\lambda}_i - \lambda_i|$ and the low-pass filtering effect becomes stronger. In contrast, decreasing γ weakens the low-pass filtering effect. Therefore, by properly tuning γ, SFNet can balance the preservation of local details and the extraction of global structure according to intrinsic characteristics of the dataset, such as uniformity of land-cover and noise level, and it can maintain robust performance in various scenarios.

a) Computational complexity: SFNet's iteration mainly involves multiplying the sparse propagation matrix P with the feature matrix, resulting in a computational complexity of $\mathcal{O}(E(A)d)$, where $E(A)$ denotes the number of edges in the superpixel graph and d is the feature dimension. Since the number of superpixels $|S|$ is much smaller than the number of pixels N, SFNet achieves efficiency that scales linearly with the number of propagation paths, making it an effective solution that balances long-range modeling capabilities and computational feasibility.

TABLE II
KEY INFORMATION OF DATASETS

Datasets	Resolution	Bands	Classes	Samples
IP	145 × 145	200	16	10,249
MU	325 × 220	64	11	53,687
SA	512 × 217	204	16	54,129
WHH	940 × 475	270	22	386,693
PU	610 × 340	103	9	42,776
HO	349 × 1905	144	15	15,029

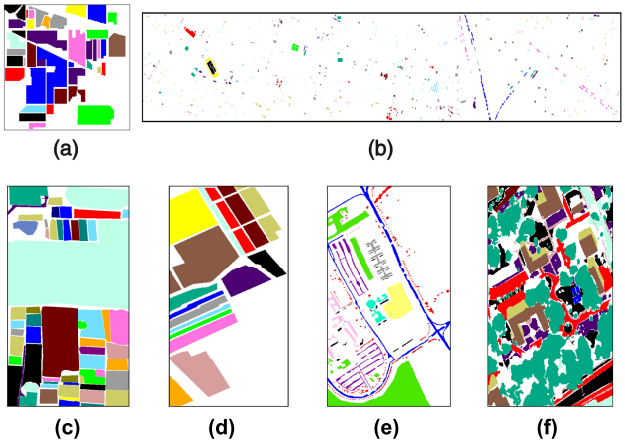


Fig. 2. Comparison of ground-truth maps for different datasets. (a) IP dataset, (b) HO dataset, (c) WHH dataset, (d) SA dataset, (e) PU dataset, and (f) MU dataset.

IV. EXPERIMENTAL RESULTS

A. Datasets and Experimental Settings

1) Datasets: To comprehensively evaluate the feature extraction capability and generalization performance of SFPNet-F/SFPNet-A, we select six representative HSI benchmarks from public datasets according to the spatial morphology of land-cover patterns and the degree of spectral mixing, including Indian Pines (IP), MUUFL (MU), Salinas (SA), WHU-Hi-HongHu (WHH), University of Pavia (PU), and Houston (HO). The key statistics of these datasets (spatial resolution, number of bands, number of classes, and sample size) are summarized in Table II, and the corresponding ground-truth maps are illustrated in Fig. 2. As can be observed, IP and MU exhibit typical spectral mixing, blurred class boundaries, and highly adjacent land-cover regions, where different categories are spatially interwoven; this poses stringent requirements on the model's discriminability over complex boundaries and locally heterogeneous areas. In contrast, SA and WHH present relatively smooth land-cover distributions with clearer block-like structures and higher intraclass spatial consistency, making them suitable for assessing a model's ability to capture region-level consistency over regular and contiguous homogeneous patches. PU and HO contain numerous irregular land-cover objects with significant scale variations, where broad interclass transition areas coexist with more pronounced background noise and mixed pixels; these datasets are therefore appropriate for testing robustness and adaptability in highly complex urban scenes.

TABLE III
KEY HYPERPARAMETER CONFIGURATIONS OF THE CORE MODULES IN THE TWO SFPNET VARIANTS

	IP	MU	SA	WHH	PU	HO
b	2×2	3×3	4×4	4×4	4×4	6×6
k	30	10	20	15	10	15
PSR	0.99	0.99	0.99	0.99	0.99	0.99
l	2	2	2	1	2	2
S	200	200	300	200	300	300
SSR	0.97	0.99	0.98	0.99	0.99	0.99
r	5	4	5	5	5	5
γ	0.90	0.90	0.00	0.20	0.1000	0.20

Moreover, to highlight the intrinsic representation capacity of different feature propagation mechanisms under a strict few-shot regime, we adopt a strongly constrained data split on all datasets: for each class, five samples are randomly selected for training and five samples for validation, while all remaining labeled samples are used for testing. This setting substantially reduces the amount of supervision, such that the resulting performance more faithfully reflects the merit of the feature propagation and representation learning mechanism itself, rather than the "data dividend" brought by abundant labeled samples.

2) Experimental Platform and Evaluation Metrics: All experiments are implemented in Python 3.10 using PyTorch 2.1.0 [60] and conducted on a workstation equipped with 25 vCPUs (AMD EPYC 7T83 64-Core Processor) and a single vGPU with 32-GB memory. Unless otherwise specified, all reported results are averaged over 10 independent runs with random seeds ranging from 0 to 9, in order to improve statistical robustness and reproducibility.

For performance evaluation, following common practice in HSI classification, we adopt overall accuracy (OA), average accuracy (AA), and the Kappa coefficient as the primary accuracy metrics to assess both overall discriminability and class-balanced performance. In addition, we report the training time, testing time, and peak GPU memory consumption on each dataset, thereby providing a system-level comparison of computational efficiency and resource usage across different feature propagation paradigms.

3) Baseline Settings: To verify the effectiveness and practicality of SFPNet-F/A, we first conduct a fair comparison against nine representative feature propagation models. These baselines cover mainstream GNN and Transformer architectures for HSI classification, including superpixel-graph feature propagation methods that leverage CNNs to produce fine-grained pixel representations, i.e., CEGCN [34], MSSGU [35], WFCG [36], AMGCFN [37], and GTFCN [40]. We further include three patch-based pixel-level feature propagation models, namely, SQSFormer [27], MASSFormer [28], and S^2GFormer [24], as well as a hybrid model, CTFN [32], which directly treats the center pixel as a token and combines it with a superpixel-level propagation mechanism.

In implementation, for each baseline we adopt the best-performing architectural variant reported in its original paper, and tune hyperparameters within the search ranges suggested by the published code or the original descriptions, aiming to reproduce their upper bound performance under the recommended settings. Considering that different methods may use different training epochs and model selection strategies, to ensure fairness and reproducibility we set the maximum number of training epochs to 500 for all methods, and follow the common practice in related works [34], [35], [36], [37] by selecting the checkpoint that achieves the minimum cross-entropy loss on the validation set for final testing. For SFPNet-F/A, we train the model using the Adam optimizer with the cross-entropy objective. The maximum number of epochs is set to 500, the initial learning rate is set to 5×10^{-4}, and the hidden feature dimension is fixed to 128. Table III summarizes the key hyperparameter configurations of the core modules in the two SFPNet variants. Specifically, the PFNet branch includes the block number b, the neighbor number k, the sparsity ratio of the induced adjacency matrix (denoted as PSR), and the number of PFNet layers l. The SFNet branch includes the superpixel segmentation scale S, the sparsity ratio of the induced adjacency matrix (denoted as SSR), the number of SFNet layers r, and the spectral decay factor γ that controls the decay behavior in the spectral domain. As shown in Table III, both neighborhood structures maintain high sparsity across datasets, which structurally supports our designation of the proposed framework as a "sparse feature propagation" approach.

Moreover, to further analyze the source of performance gains of SFPNet-F/A and to validate the contribution of its core modules (PFNet-F/PFNet-A and SFNet), we construct a set of single-branch or simplified counterparts under the same training protocol and comparable hyperparameter settings, and perform systematic comparisons with SFPNet-F/A. These counterparts include: 1) a conventional SVM classifier; 2) a denoising-and-grouping CNN backbone that has been widely adopted in [32], [34], [35], [36], and [40]; 3) a superpixel-level GCN-S that performs one-hop neighborhood propagation on the superpixel graph [15]; 4) a pixel-level Transformer-P with spatial coordinates as relative positional encodings [18] and a superpixel-level Transformer-S using distances between superpixel centroids as relative positional encodings; and 5) Linformer-P [53] and Linformer-S that employ an improved linear-attention mechanism at the pixel and superpixel levels, respectively; and the P-ViT based on the Visual Transformer [27].

Except for SVM, all these counterparts are implemented under the same training protocol and comparable hyperparameter search space as SFPNet. In particular, the maximum number of training epochs is set to 500, the initial learning rate is set to 5×10^{-4}, and the hidden dimension is fixed to 128. The number of layers is searched within $\{2, 3, 4, 5\}$. For attention-based models, the number of attention heads is searched in $\{2, 4, 8\}$. For the vision Transformer variant, the patch size is selected from $\{9 \times 9, 11 \times 11, 13 \times 13\}$. For Linformer-P/S, the projection dimension (i.e., the focusing-function hyperparameter) P is searched within $\{1, 2, 3\}$.

B. Comparison With Baseline Methods

To comprehensively evaluate the overall performance of the proposed SFPNet framework, we conduct systematic comparisons with nine state-of-the-art GNN- and Transformer-based feature propagation models on six public HSI benchmarks, from three perspectives: classification accuracy, computational efficiency, and multiscale land-cover modeling capability.

1) Comparison of Classification Accuracy: The OA, AA, and Kappa coefficient are reported in Table IV. Both SFPNet variants (SFPNet-F and SFPNet-A) consistently and substantially outperform all baselines across all six datasets on all three core metrics. Specifically, compared with the best baseline (i.e., the datasetwise SOTA competitor), SFPNet-F achieves average improvements of 1.98%, 2.13%, and 2.70% in OA, AA, and Kappa, respectively. With the introduction of dynamic attention weights, SFPNet-A further yields larger average gains of 2.25%, 2.48%, and 3.02% on the corresponding metrics.

These results validate the effectiveness of SFPNet in jointly leveraging the fine-grained modeling capability of the pixel-level PFNet branch and the long-range dependency modeling capability of the superpixel-level SFNet branch under a unified sparse propagation paradigm. In particular, on the IP and MU datasets, which exhibit higher spectral–spatial heterogeneity, SFPNet-A shows a more pronounced advantage over SFPNet-F, indicating that dynamic weighting can capture contextual dependencies more precisely in complex scenes.

2) Computational Efficiency and Resource Consumption: To clearly present the practical computational cost of different models, Table V reports the training time, testing time, and peak GPU memory footprint of each method on the six datasets under the classification settings corresponding to Table IV. Note that different methods do not follow exactly the same training and inference paradigms. Inductive methods, such as SQSFormer [27], MASSFormer [28], S^2GFormer [24], and CTFN [32], usually construct local patches centered on training pixels and process these local samples in each training epoch. In contrast, the proposed SFPNet-F/A, together with CEGCN [34], MSSGU [35], WFCG [36], AMGCFN [37], and GTCFN [40], follows a paradigm of transductive feature propagation, where feature propagation is performed throughout the image or the entire graph structure in each iteration. Therefore, the runtime and memory results in Table V mainly reflect the end-to-end practical computational cost of each method under its recommended training and inference paradigm, rather than a per-iteration computational cost comparison under exactly the same computational unit or the same per-epoch sample scale.

Under this interpretation, the joint analysis of efficiency and accuracy shows that SFPNet-F exhibits favorable training efficiency among transductive feature propagation models, with a shorter training time than most feature propagation models. Although inductive pixel-level models such as SQSFormer, MASSFormer, and S^2 GFormer can reduce the computational scale on the training-side by processing only local patches centered on training pixels in each epoch, especially when the foreground samples are sparse, Table IV shows that the mean

TABLE IV

COMPARISON OF OVERALL CLASSIFICATION PERFORMANCE ACROSS SIX DATASETS FOR DIFFERENT METHODS, WHERE THE BEST RESULT IS DISPLAYED IN RED, THE SECOND-BEST IN BLUE, AND THE THIRD-BEST IN BOLD BLACK

Dataset	Perf.	CEGCN	MSSGU	WFCG	AMGCFN	GTFCN	SQSformer	MASSformer	S2GFormer	CTFN	SFPNet-F	SFPNet-A
IP	OA	73.65 ± 2.75	**79.31 ± 3.39**	78.31 ± 2.64	78.56 ± 3.27	36.56 ± 9.58	78.41 ± 2.10	75.87 ± 3.08	71.34 ± 2.49	73.77 ± 1.67	81.62 ± 2.43	81.93 ± 2.43
	AA	84.11 ± 1.45	86.14 ± 2.21	87.23 ± 1.81	86.69 ± 1.50	52.32 ± 6.65	**87.91 ± 1.23**	85.30 ± 1.82	82.89 ± 1.54	83.37 ± 1.90	88.69 ± 1.49	89.08 ± 1.33
	Kpp	70.35 ± 2.94	**76.32 ± 3.84**	75.71 ± 2.94	75.82 ± 3.62	28.94 ± 9.11	76.08 ± 2.33	72.82 ± 3.37	67.89 ± 2.72	70.25 ± 1.86	79.19 ± 2.72	79.56 ± 2.69
MU	OA	67.72 ± 3.74	60.46 ± 3.07	**72.46 ± 3.72**	64.18 ± 4.45	37.82 ± 18.66	64.37 ± 3.12	67.37 ± 2.37	61.08 ± 3.88	68.77 ± 3.60	73.46 ± 2.56	74.10 ± 3.47
	AA	66.85 ± 4.22	65.26 ± 1.71	**72.03 ± 3.58**	65.62 ± 5.66	26.71 ± 10.12	60.42 ± 2.82	70.30 ± 2.18	61.62 ± 2.95	67.48 ± 3.28	75.74 ± 2.51	76.72 ± 3.00
	Kpp	59.48 ± 4.42	51.65 ± 3.18	**65.34 ± 4.25**	55.87 ± 4.77	18.03 ± 13.74	55.94 ± 3.58	59.39 ± 2.71	52.58 ± 4.46	60.94 ± 4.07	67.02 ± 2.92	67.80 ± 3.91
SA	OA	91.46 ± 3.48	93.52 ± 1.21	**95.54 ± 1.74**	93.87 ± 1.27	66.25 ± 10.63	92.43 ± 1.07	92.89 ± 1.95	90.50 ± 1.59	87.63 ± 3.26	98.17 ± 1.26	98.25 ± 1.02
	AA	96.05 ± 0.91	95.06 ± 0.92	**96.99 ± 0.84**	96.59 ± 0.45	70.86 ± 8.69	96.30 ± 0.56	96.19 ± 0.90	93.18 ± 1.26	94.42 ± 1.27	98.62 ± 0.46	98.60 ± 0.37
	Kpp	90.54 ± 3.82	92.77 ± 1.36	92.82 ± 1.93	**93.19 ± 1.41**	62.38 ± 11.35	91.58 ± 1.19	92.09 ± 2.16	89.45 ± 1.76	86.32 ± 3.57	97.96 ± 1.40	98.05 ± 1.13
WHH	OA	83.56 ± 1.73	84.11 ± 2.87	83.66 ± 3.48	**84.94 ± 3.35**	23.43 ± 20.44	81.17 ± 2.47	75.17 ± 5.68	73.14 ± 10.82	80.20 ± 3.38	86.70 ± 2.54	86.94 ± 1.69
	AA	81.42 ± 1.12	82.07 ± 2.47	81.92 ± 1.77	**82.70 ± 2.47**	24.17 ± 6.15	80.61 ± 1.62	73.28 ± 2.83	78.13 ± 9.15	75.21 ± 1.96	86.12 ± 1.74	86.15 ± 1.54
	Kpp	79.69 ± 1.98	80.27 ± 3.33	79.94 ± 3.95	**81.41 ± 3.81**	15.39 ± 12.17	76.88 ± 2.76	70.14 ± 5.98	75.31 ± 12.67	75.63 ± 3.77	83.53 ± 2.94	83.77 ± 1.99
PU	OA	**86.55 ± 4.88**	82.02 ± 8.64	86.40 ± 4.32	82.57 ± 5.98	52.28 ± 20.21	75.58 ± 6.71	77.58 ± 5.52	69.95 ± 4.77	75.44 ± 7.97	88.36 ± 3.28	88.59 ± 3.56
	AA	91.62 ± 2.67	90.22 ± 3.42	**92.27 ± 2.06**	88.49 ± 2.72	48.81 ± 19.23	83.18 ± 1.96	83.57 ± 3.17	77.97 ± 2.24	82.46 ± 2.27	93.62 ± 1.72	94.31 ± 1.47
	Kpp	**82.95 ± 5.81**	77.81 ± 9.92	82.79 ± 5.12	78.03 ± 6.91	36.04 ± 22.66	69.38 ± 7.57	71.71 ± 6.43	62.47 ± 5.28	69.15 ± 8.73	85.17 ± 4.00	85.46 ± 4.32
HO	OA	**78.66 ± 3.36**	71.43 ± 2.64	77.59 ± 3.63	**77.77 ± 2.97**	44.29 ± 9.65	76.50 ± 2.76	78.45 ± 2.89	75.61 ± 2.88	78.18 ± 2.44	81.00 ± 2.96	81.19 ± 2.78
	AA	80.14 ± 3.17	75.63 ± 2.25	80.14 ± 3.24	80.74 ± 2.71	45.88 ± 11.72	79.85 ± 2.35	81.91 ± 2.51	79.11 ± 2.42	81.26 ± 2.15	83.77 ± 2.54	83.84 ± 2.44
	Kpp	**76.92 ± 3.64**	69.15 ± 2.83	75.77 ± 3.92	75.97 ± 3.21	39.52 ± 10.56	74.61 ± 2.98	76.72 ± 3.13	73.66 ± 3.10	76.42 ± 2.65	79.48 ± 3.20	79.63 ± 3.00

TABLE V

COMPARISON OF TRAINING TIME, TESTING TIME, AND PEAK GPU MEMORY USAGE FOR EACH METHOD ACROSS THE SIX DATASETS UNDER THE CLASSIFICATION PERFORMANCE METRICS IN TABLE IV

Dataset	Efficiency	CEGCN	MSSGU	WFCG	AMGCFN	GTCFN	SQSformer	MASSformer	S2GFormer	CTFN	SFPNet-F	SFPNet-A
IP	Train time	10.03s	13.65s	13.76s	25.17s	27.02s	26.16s	19.70s	45.56s	35.59s	19.46s	45.16s
	Test time	0.18s	0.20s	0.19s	0.21s	0.20s	4.74s	1.13s	3.06s	2.04s	0.04s	0.05s
	Memory	0.28GB	0.35GB	0.32GB	0.53GB	0.35GB	0.91GB	0.05GB	0.11GB	0.52GB	0.54GB	2.44GB
MU	Train time	11.90s	15.29s	29.81s	41.84s	32.30s	15.73s	14.16s	29.96s	19.73s	22.11s	53.59s
	Test time	0.18s	0.20s	0.21s	0.22s	0.19s	18.52s	14.64s	16.54s	13.64s	0.07s	0.08s
	Memory	0.56GB	0.52GB	1.56GB	1.53GB	0.81GB	0.82GB	0.05GB	0.10GB	1.32GB	0.76GB	2.94GB
SA	Train time	21.31s	30.78s	61.63s	68.08s	40.83s	94.15s	21.59s	45.96s	54.63s	47.53s	120.87s
	Test time	0.24s	0.25s	0.26s	0.26s	0.24s	149.51s	15.31s	16.77s	23.22s	0.08s	0.12s
	Memory	1.01GB	0.82GB	3.47GB	2.75GB	1.30GB	2.31GB	0.06GB	0.12GB	2.51GB	2.15GB	8.98GB
WHH	Train time	81.94s	94.43s	281.09s	238.38s	123.44s	70.24s	37.08s	64.98s	334.09s	60.9s	122.59s
	Test time	0.55s	0.56s	0.71s	0.61s	0.54s	290.55s	59.83s	118.93s	902.52s	0.30s	0.32s
	Memory	4.27GB	2.76GB	14.41GB	11.59GB	5.11GB	2.31GB	0.17GB	0.14GB	10.92GB	3.99GB	10.82GB
PU	Train time	45.61s	49.97s	142.59s	126.09s	65.12s	27.48s	14.73s	29.41s	52.13s	49.40s	126.80s
	Test time	0.28s	0.30s	0.34s	0.33s	0.31s	79.21s	8.40s	23.02s	33.25s	0.08s	0.11s
	Memory	1.79GB	1.14GB	10.05GB	4.61GB	2.17GB	1.59GB	0.05GB	0.10GB	3.97GB	2.19GB	8.51GB
HO	Train time	121.89s	183.61s	362.34s	343.27s	180.30s	35.67s	21.35s	45.13s	479.61s	71.89s	172.58s
	Test time	0.32s	0.33s	0.52s	0.41s	0.35s	166.66s	12.78s	14.47s	65.60s	0.07s	0.13s
	Memory	5.61GB	3.41GB	16.64GB	15.19GB	6.97GB	1.53GB	0.11GB	0.16GB	13.78GB	3.92GB	14.31GB

OA of SFPNet-F over the six datasets is, on average, 5.41% higher than the best result among these three inductive models. This indicates that SFPNet-F maintains competitive practical training efficiency while significantly improving classification performance. SFPNet-A requires a longer training time due to the introduction of dynamic attention weights; nevertheless, compared with structurally complex baseline models such as WFCG and AMGCFN, its computational cost remains within an acceptable range while providing higher classification accuracy in complex scenes.

Second, in terms of inference efficiency, SFPNet and other transductive feature propagation models can complete full-image feature updating and prediction within one forward pass, thus avoiding the repeated patchwise inference required by inductive patch-based methods over a large number of testing pixels. Therefore, in the experimental setting of this study, transductive methods usually exhibit a higher full-scene inference efficiency. Benefiting from the sparse feature propagation design, SFPNet-F/A avoids global dense computation during the forward pass, and its testing time is lower than that of most baseline methods. In particular, compared to inductive feature propagation models such as SQSFormer, MASSFormer, S^2GFormer, and CTFN, SFPNet-F/A shows a more prominent advantage in inference efficiency. For example, on the WHH dataset with relatively dense land-cover distributions, the per-run testing time of the above inductive models can even exceed their training time, whereas SFPNet-F/A still maintains efficient full-scene inference.

TABLE VI

COMPARISON OF MULTI-SCALE LAND-COVER CLASSIFICATION PERFORMANCE ACROSS SIX DATASETS FOR DIFFERENT METHODS, WHERE SAA, MAA, AND LAA DENOTE SMALL-SCALE, MEDIUM-SCALE, AND LARGE-SCALE AVERAGE ACCURACY, RESPECTIVELY. THE BEST RESULT IS DISPLAYED IN RED, THE SECOND-BEST IN BLUE, AND THE THIRD-BEST IN BOLD BLACK

Dataset	Acc.	CEGCN	MSSGU	WFCG	AMGCFN	GTFCN	SQSformer	MASSformer	S2GFormer	CTFN	SFPNet-F	SFPNet-A
IP	SAA	99.33 ± 0.52	99.09 ± 0.98	99.07 ± 0.84	99.07 ± 0.83	85.41 ± 8.43	**99.43 ± 0.53**	99.15 ± 0.86	98.70 ± 1.56	99.04 ± 0.85	99.49 ± 0.42	99.57 ± 0.58
	LAA	72.66 ± 15.58	79.01 ± 13.55	79.08 ± 14.01	81.95 ± 14.47	40.07 ± 24.44	**79.39 ± 12.58**	78.21 ± 13.38	73.32 ± 18.16	77.44 ± 17.23	81.95 ± 11.45	83.01 ± 11.35
	MAA	86.15 ± 10.64	85.79 ± 9.17	92.32 ± 9.22	88.91 ± 13.40	45.46 ± 12.40	91.22 ± 8.61	86.58 ± 12.45	83.39 ± 13.89	83.50 ± 13.79	85.67 ± 10.09	**89.20 ± 9.70**
MU	SAA	66.82 ± 27.59	69.81 ± 24.54	**72.56 ± 19.54**	67.51 ± 20.78	25.02 ± 12.76	69.64 ± 23.30	72.19 ± 20.99	65.58 ± 23.42	66.50 ± 20.44	75.00 ± 16.75	76.24 ± 16.68
	LAA	62.91 ± 12.55	53.58 ± 11.25	**68.43 ± 10.46**	58.09 ± 10.76	28.74 ± 19.58	57.09 ± 11.35	64.50 ± 9.15	53.83 ± 12.30	62.60 ± 11.46	71.08 ± 6.86	71.17 ± 6.53
	MAA	72.16 ± 7.34	74.44 ± 9.67	75.14 ± 3.10	73.13 ± 8.32	26.26 ± 1.98	74.57 ± 6.68	**75.30 ± 3.86**	66.74 ± 1.96	**75.30 ± 2.90**	82.94 ± 4.13	84.77 ± 5.06
SA	SAA	98.68 ± 1.15	96.72 ± 2.52	98.21 ± 0.98	98.55 ± 1.12	76.08 ± 10.01	**98.59 ± 1.49**	97.96 ± 2.27	92.82 ± 4.23	97.58 ± 2.33	98.70 ± 0.83	98.68 ± 1.06
	LAA	93.17 ± 11.36	93.78 ± 8.03	**95.67 ± 8.33**	95.13 ± 6.78	68.18 ± 14.23	93.14 ± 9.98	93.79 ± 9.19	92.90 ± 9.69	90.35 ± 17.76	98.78 ± 1.77	99.02 ± 1.66
	MAA	96.88 ± 4.44	94.95 ± 4.31	**97.55 ± 3.15**	96.38 ± 4.47	68.87 ± 13.94	97.81 ± 2.21	97.30 ± 2.49	93.87 ± 2.81	96.14 ± 4.36	98.14 ± 2.75	96.22 ± 4.49
WHH	SAA	87.00 ± 15.44	**90.06 ± 8.08**	85.91 ± 10.82	86.12 ± 13.63	29.14 ± 15.50	86.86 ± 10.99	76.59 ± 12.50	84.18 ± 15.48	77.20 ± 16.11	91.62 ± 5.26	91.53 ± 5.69
	LAA	76.89 ± 12.95	77.42 ± 12.74	79.67 ± 10.56	**80.10 ± 9.19**	20.85 ± 10.09	74.20 ± 15.46	70.62 ± 14.87	69.44 ± 17.79	75.28 ± 10.73	81.38 ± 9.15	82.77 ± 8.00
	MAA	79.58 ± 13.82	77.60 ± 13.32	79.62 ± 9.39	**81.39 ± 9.56**	21.82 ± 5.25	79.88 ± 14.69	72.16 ± 16.75	71.06 ± 19.28	72.87 ± 11.49	84.69 ± 11.01	84.43 ± 11.37
PU	SAA	96.18 ± 5.68	97.52 ± 2.74	97.64 ± 3.73	96.84 ± 2.64	62.97 ± 5.37	97.88 ± 2.41	95.59 ± 5.83	**98.55 ± 1.31**	95.99 ± 4.33	99.07 ± 1.98	99.43 ± 0.86
	LAA	**87.32 ± 7.85**	83.09 ± 13.82	86.51 ± 8.32	81.55 ± 10.80	49.77 ± 12.33	71.31 ± 11.72	76.15 ± 10.59	67.12 ± 19.24	70.43 ± 4.56	87.95 ± 7.68	87.44 ± 8.10
	MAA	91.38 ± 4.69	90.04 ± 3.65	**92.33 ± 1.42**	87.07 ± 3.62	34.69 ± 22.64	80.49 ± 7.36	79.30 ± 9.18	70.91 ± 17.31	81.60 ± 9.60	93.84 ± 4.63	95.72 ± 2.67
HO	SAA	87.37 ± 16.60	93.47 ± 8.63	91.53 ± 12.71	93.43 ± 5.25	55.05 ± 26.28	94.28 ± 6.88	**95.47 ± 4.35**	93.97 ± 7.69	94.45 ± 4.12	95.75 ± 5.65	95.81 ± 5.61
	LAA	77.66 ± 7.26	62.98 ± 11.64	77.33 ± 7.28	76.17 ± 6.88	45.73 ± 13.98	71.90 ± 12.65	75.63 ± 12.85	70.55 ± 14.06	**81.03 ± 4.44**	82.80 ± 5.79	84.06 ± 5.22
	MAA	75.77 ± 17.91	69.19 ± 11.94	72.54 ± 16.67	73.21 ± 15.34	38.33 ± 15.62	73.12 ± 17.48	75.30 ± 17.79	72.45 ± 17.25	70.42 ± 19.11	**74.44 ± 17.50**	73.88 ± 18.38

In addition, in terms of resource usage, SFPNet-F has a moderate peak GPU memory footprint across the six datasets, indicating a good balance between computational efficiency and memory demand. SFPNet-A requires relatively higher memory due to the additional parameterization and computational overhead introduced by dynamic attention weights. In general, when jointly considering classification accuracy, training time, testing time, and memory consumption, SFPNet-F achieves a favorable balance among performance improvement, inference efficiency, and resource consumption, making it suitable for scenarios sensitive to inference latency and resource cost. In contrast, SFPNet-A provides a higher accuracy ceiling for complex land-cover scenes at the cost of controllably increasing computational resources.

3) Multiscale Land-Cover Modeling Capability: Large-scale land-cover regions typically exhibit stronger spectral and spatial consistency with relatively lower noise, whereas small-scale regions are more likely to appear near class boundaries or complex backgrounds, where multiple categories are mixed in the neighborhood. Such small-scale targets impose more stringent requirements on a model's capability of capturing fine-grained dependencies. To systematically evaluate how different methods characterize land-cover categories at different spatial scales, we apply a logarithmic transform to the number of pixels in each class and use the 33rd and 67th percentiles as thresholds to split classes into three scale groups (small, medium, and large). We then compute the corresponding mean per-class accuracy for each group, denoted as small-scale AA (SAA), medium-scale AA (MAA), and large-scale AA (LAA), respectively.

The results are summarized in Table VI. Across all datasets, both SFPNet variants achieve overall superior performance on small-scale (SAA) and large-scale (LAA) land-cover categories compared with all baseline methods. In particular, relative to the best baseline on each dataset, SFPNet-A achieves average improvements of 1.18% and 2.72% on SAA and LAA, respectively, while SFPNet-F obtains corresponding gains of 1.11% and 2.31%. These results indicate that, through the dual-path sparse collaboration between the pixel-level PFNet branch and the superpixel-level SFNet branch, SFPNet can simultaneously strengthen fine-grained characterization of local structures and enhance long-range dependency modeling over large-scale regions within a unified framework. Moreover, for medium-scale categories (MAA), both SFPNet variants remain leading or near-leading on most datasets, demonstrating favorable cross-scale generalization.

By further considering dataset-specific land-cover distributions and the architectural designs of different baselines, we observe several phenomena. First, on datasets with a higher proportion of large homogeneous regions, such as WHH and PU, the LAA values of four patch-based pixel-level feature propagation models (SQSFormer, MASSFormer, S^2GFormer, and CTFN) are generally lower than those of representative superpixel-level propagation models (e.g., CEGCN, MSSGU, WFCG, and AMGCFN). In contrast, on the HO dataset where land-cover regions are typically smaller and more spatially fragmented, patch-based methods tend to achieve better SAA than the above superpixel-level approaches. This comparison suggests that patch-based strategies can reasonably cover small targets and control noise in scenes with relatively small regions and simpler backgrounds. However, in scenarios containing large homogeneous areas or complex interclass boundaries, their limited receptive fields are insufficient to capture long-range dependencies and may introduce more noise in densely distributed regions. By contrast, pixels within a superpixel are often more homogeneous, which facilitates more stable aggregation in large-scale regions.

Second, although GTFCN incorporates a superpixel-level feature propagation mechanism, its overall performance still lags behind other methods by a large margin. A plausible explanation is that its global attention mechanism is less effective in suppressing global noise under extremely limited supervision, which will be further discussed in the next subsection. In comparison, both SFPNet variants exhibit more balanced performance in small-, medium-, and large-scale categories, without showing an obvious preference for small or large targets. In particular, the dynamic attention mechanism in SFPNet-A provides an adaptive focusing ability

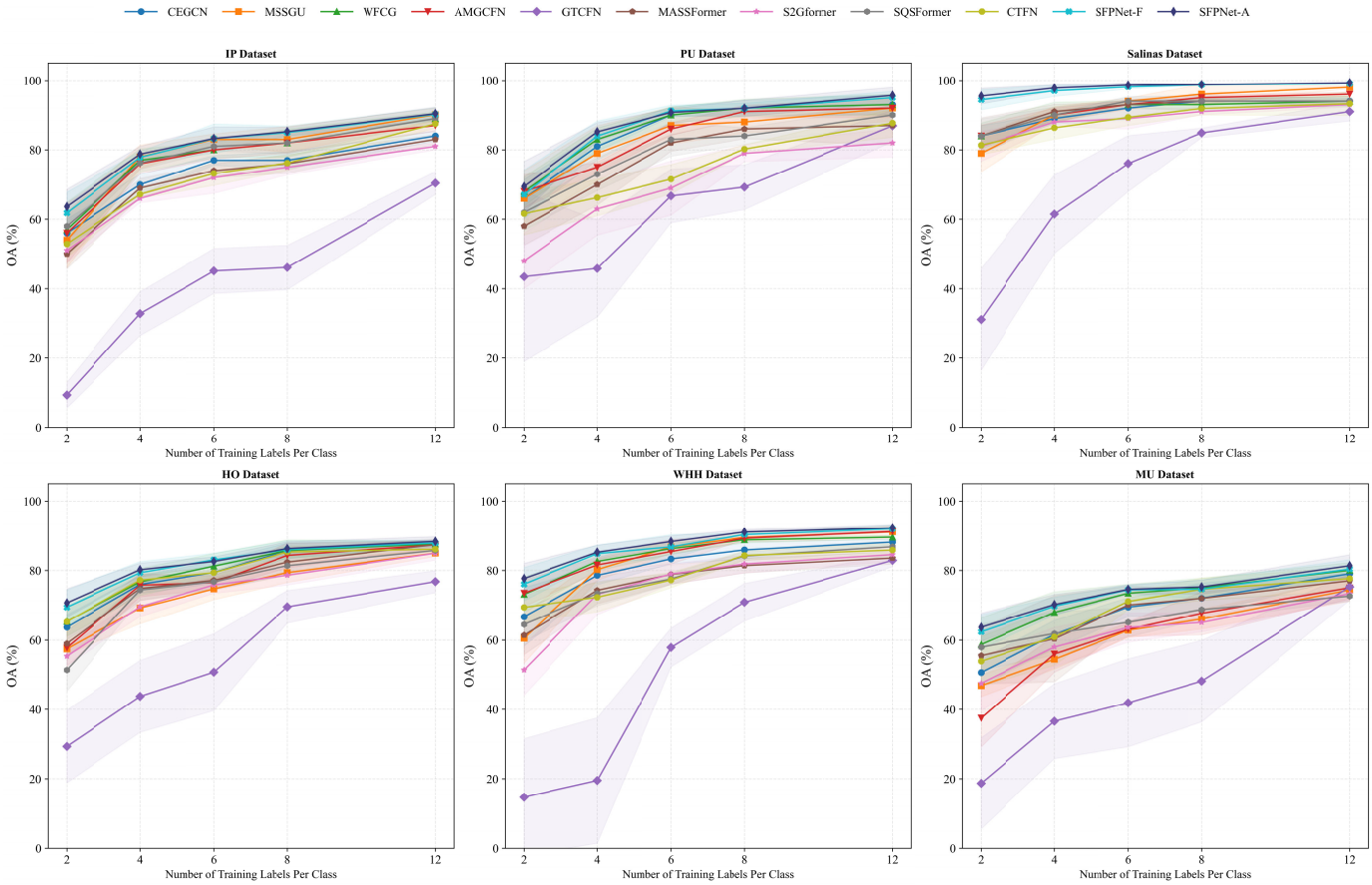


Fig. 3. Comparison of overall classification accuracy (OA) across different methods at various training sample sizes.

on critical contextual nodes, which in most cases leads to a higher SAA than SFPNet-F and further strengthens fine-grained representation learning.

In summary, the two proposed SFPNet variants not only achieve significant improvements in overall classification accuracy, but also demonstrate comprehensive advantages in computational efficiency, multiscale adaptability, and robustness to limited supervision. More importantly, they consistently maintain leading and well-balanced performance across land-cover categories with different spatial distributions and scales, without evident scale bias or scene dependency, which reflects strong generalization capability.

To further verify that these gains are not due to incidental effects from a particular data split, we conduct controlled ablation experiments by varying the number of labeled training samples per class within $\{2, 4, 6, 8, 12\}$. Performance trends at different levels of supervision are shown in Fig. 3. The results clearly indicate that the superpixel-level model GTFCN, which relies on global attention, is highly dependent on abundant labeled samples and suffers a sharp performance drop under scarce supervision. In addition, many baselines exhibit considerable fluctuations across datasets with different land-cover patterns. In contrast, both SFPNet variants consistently stay within the best-performing regime on all datasets and under all label configurations, demonstrating superior robustness. Notably, on datasets with relatively simpler land-cover structures, such as SA, HO, and WHH, SFPNet exhibits smaller marginal gains as the number of labeled samples increases. This observation suggests that, owing to its efficient sparse feature propagation architecture, SFPNet is able to effectively exploit intrinsic spectral–spatial structures from very limited supervision, achieving label-efficient learning and strong inference capability. Collectively, these controlled experiments confirm that the superior performance of the two SFPNet variants stems from their inherent architectural generalization and stability, rather than from coincidental factors under specific experimental settings.

C. Ablation Study

The preceding experiments have systematically demonstrated the effectiveness of the SFPNet variants in modeling fine-grained local dependencies and long-range contextual dependencies. In order to further trace the source of the performance gains and analyze the mechanisms of the core designs, we conduct a series of ablation studies in this section. The analysis is progressively organized from three aspects. First, the core modules in the proposed framework are treated as independent classification units and compared with several representative baselines of a single module to quantify the intrinsic representation capacity of each module. Second, based on single-module analysis, we further compare different fusion strategies, including fixed weighted

TABLE VII

COMPARISON OF MODULE-LEVEL CLASSIFICATION PERFORMANCE ACROSS SIX DATASETS FOR DIFFERENT SINGLE-MODULE METHODS, WHERE THE BEST RESULT IS DISPLAYED IN RED, THE SECOND- BEST IN BLUE, AND THE THIRD-BEST IN BOLD BLACK

Dataset	Performance	SVM	CNN	S-Transformer	S-Linformer	S-GCN	P-Transformer	P-Linformer	P-VIT	SFNet	PFNet-F	PFNet-A
IP	OA	35.87 ± 6.68	64.84 ± 3.97	64.87 ± 1.82	38.05 ± 6.50	73.04 ± 2.30	43.59 ± 3.40	34.12 ± 11.38	64.50 ± 2.81	**73.05 ± 3.19**	73.80 ± 1.29	75.10 ± 2.00
	SAA	87.11 ± 7.85	**97.93 ± 3.32**	82.55 ± 23.72	69.24 ± 32.66	92.12 ± 14.67	84.18 ± 7.52	54.87 ± 16.70	88.63 ± 5.64	92.13 ± 14.58	98.59 ± 0.50	99.68 ± 0.69
	LAA	36.39 ± 17.32	59.26 ± 20.43	58.11 ± 17.74	31.78 ± 7.43	71.39 ± 14.22	37.35 ± 15.22	33.72 ± 9.18	60.60 ± 17.06	73.17 ± 14.17	69.49 ± 12.26	**71.02 ± 10.56**
MU	OA	44.53 ± 5.24	71.51 ± 2.45	38.37 ± 6.45	26.69 ± 11.25	47.45 ± 4.09	*OOM*	27.87 ± 17.92	59.74 ± 3.94	55.03 ± 3.52	**71.45 ± 2.97**	72.47 ± 2.61
	SAA	70.13 ± 17.79	66.00 ± 31.52	55.26 ± 31.45	50.20 ± 33.28	**70.17 ± 30.19**	*OOM*	24.60 ± 18.62	59.10 ± 41.55	66.36 ± 34.82	70.67 ± 21.21	72.22 ± 16.67
	LAA	40.84 ± 28.23	66.26 ± 13.65	28.39 ± 14.65	23.09 ± 5.86	65.95 ± 11.31	*OOM*	23.78 ± 11.67	57.16 ± 5.52	68.90 ± 14.87	**67.38 ± 5.92**	68.19 ± 4.38
SA	OA	73.59 ± 2.03	88.34 ± 4.87	78.24 ± 4.57	72.99 ± 4.98	95.05 ± 1.11	*OOM*	69.82 ± 16.30	91.26 ± 1.60	96.15 ± 1.81	92.08 ± 1.50	**92.51 ± 1.32**
	SAA	89.10 ± 8.55	97.35 ± 3.81	80.21 ± 13.61	66.30 ± 16.52	**97.88 ± 3.12**	*OOM*	71.93 ± 21.75	95.24 ± 2.44	95.17 ± 5.21	98.01 ± 1.26	98.07 ± 1.11
	LAA	82.26 ± 19.94	90.93 ± 16.96	84.75 ± 15.93	80.28 ± 7.21	94.28 ± 3.75	*OOM*	73.94 ± 14.75	92.61 ± 9.85	95.34 ± 4.60	91.52 ± 12.56	**93.01 ± 10.44**
WHH	OA	40.22 ± 6.31	68.86 ± 3.66	**79.33 ± 3.61**	39.47 ± 23.23	83.57 ± 3.45	*OOM*	18.45 ± 17.32	71.28 ± 11.47	84.14 ± 1.32	75.61 ± 3.92	76.71 ± 4.30
	SAA	37.62 ± 21.72	69.43 ± 23.60	74.93 ± 15.46	57.67 ± 13.80	90.64 ± 6.94	*OOM*	17.35 ± 10.98	73.12 ± 17.30	90.84 ± 7.38	74.91 ± 16.96	**75.56 ± 16.84**
	LAA	38.88 ± 17.82	60.75 ± 25.03	**71.70 ± 15.46**	45.89 ± 17.00	79.96 ± 9.40	*OOM*	18.01 ± 12.87	63.20 ± 19.56	80.42 ± 10.43	63.96 ± 17.77	64.25 ± 18.53
PU	OA	63.44 ± 3.26	**77.96 ± 4.87**	67.09 ± 4.69	38.25 ± 20.57	73.79 ± 3.70	*OOM*	47.51 ± 21.49	70.65 ± 9.41	74.15 ± 6.50	80.13 ± 5.47	80.50 ± 4.39
	SAA	94.11 ± 5.18	**97.23 ± 1.62**	71.62 ± 24.27	34.18 ± 15.05	75.52 ± 32.44	*OOM*	88.54 ± 10.18	91.24 ± 9.06	71.67 ± 37.26	97.52 ± 3.16	98.35 ± 2.38
	LAA	46.33 ± 26.22	70.59 ± 11.82	60.88 ± 22.20	47.24 ± 7.73	**73.67 ± 23.07**	*OOM*	43.41 ± 12.55	66.43 ± 9.67	75.31 ± 21.81	73.61 ± 2.69	74.55 ± 3.37
HO	OA	54.95 ± 1.87	**74.7 ± 1.82**	54.64 ± 6.16	33.59 ± 13.13	62.72 ± 2.92	*OOM*	25.13 ± 15.38	76.26 ± 2.70	63.08 ± 2.95	75.37 ± 2.12	75.66 ± 2.77
	SAA	76.25 ± 36.86	83.60 ± 30.15	66.60 ± 15.13	57.27 ± 13.80	**90.82 ± 9.13**	*OOM*	41.61 ± 9.64	91.16 ± 12.37	88.26 ± 12.10	94.89 ± 6.41	94.89 ± 6.09
	LAA	62.79 ± 20.75	**77.03 ± 15.19**	53.42 ± 15.20	22.81 ± 13.38	52.39 ± 12.88	*OOM*	26.61 ± 8.42	75.85 ± 8.89	63.97 ± 9.63	79.52 ± 9.97	79.88 ± 10.50

fusion, gated fusion, and additive fusion, to evaluate the influence of core modules and their fusion on classification performance. Finally, mechanism-oriented visualizations and diagnostic experiments are conducted to reveal why these modules are effective.

1) Module Performance Analysis: To fairly evaluate the basic performance of each submodule, we keep the hyperparameters of the corresponding modules consistent with those used in the full-SFPNet framework, while equipping PFNet-F/PFNet-A and SFNet with independent classifiers, respectively. We then compare them with eight representative single-module methods. The OA, SAA, and LAA results on the six datasets are reported in Table VII.

First, on datasets with complex land-cover distributions and strong spatial heterogeneity, including IP, MU, and PU, PFNet-F/PFNet-A achieves significantly better OA and SAA than the other baseline methods. This indicates that the sparse neighborhood constructed based on spectral–spatial similarity can effectively guide feature propagation among semantically similar pixels, thereby producing highly discriminative fine-grained representations. In particular, the dynamic attention mechanism in PFNet-A further refines contextual selection, leading to better performance than the fixed-weight PFNet-F and other baselines. In contrast, on datasets dominated by regular and homogeneous land-cover regions, such as SA and WHH, SFNet achieves significantly higher OA than the other methods. It also obtains the best LAA on all datasets except for HO. These results verify that the decay aggregation and state preservation mechanisms of SFNet can effectively capture long-range dependency representations at the superpixel level.

We also observe that Transformer models based on global dense attention, namely, P-/S-Transformer [18], have difficulty filtering heterogeneous noise in complex scenes, resulting in unsatisfactory performance. Their variants using linear attention kernels, namely, P-/S-Linformer [53], suffer further performance degradation because they lack the feature-focusing capability induced by the Softmax operator. patch-based P-ViT [27] partitions local regions centered on each pixel, thus avoiding the interference of a large amount of irrelevant noise. As a result, it achieves a competitive OA on the HO dataset, where the foreground pixels are relatively limited and sparsely distributed. However, its SAA and LAA are still significantly lower than those of the PFNet modules. This suggests that the proposed sparse propagation strategy achieves a better balance between suppressing noise propagation and preserving feature propagation connectivity, as guaranteed by Theorem 1, thus effectively alleviating the inherent receptive-field limitation of patch-based strategies.

In addition, Fig. 4 reports the runtime cost, including training time and testing time, of each single-module method under the setting corresponding to Table VII. The results show that SFNet achieves an efficiency level comparable to the fastest deep graph learning baseline, GCN-S, indicating that SFNet can effectively capture long-range dependencies without noticeably increasing the overall computational burden. Using the sparse feature propagation mechanism, PFNet-F achieves significantly lower average runtime than pixel-level inference models such as CNN, P-Linformer, and P-ViT across the six datasets. Although PFNet-A introduces additional linear projections for attention computation and, therefore, requires a longer runtime, it still shows a clear efficiency advantage over the dense pixel-level self-attention model P-Transformer. These observations also indicate that the computational cost of SFPNet-F/SFPNet-A is mainly dominated by the PFNet-F/PFNet-A branch. Overall, PFNet achieves strong fine-grained modeling capability while maintaining the competitive efficiency through sparsification, whereas SFNet provides powerful long-range dependency modeling capability with near-optimal graph learning efficiency.

2) Analysis of Feature Fusion Mechanisms: To further evaluate the contribution of the parallel fusion mechanism in (7), we conduct additional ablation experiments with different fusion strategies on the six datasets, as reported in Table VIII. Specifically, we compare SFNet alone, PFNet-F alone, weighted fusion with different fixed coefficients, gated fusion, and the parallel additive fusion adopted by SFPNet-F. For fixed weighted fusion, the fused feature is defined as $Z = \alpha Z_P + (1 - \alpha) Z_S$, where Z_P and Z_S denote the output

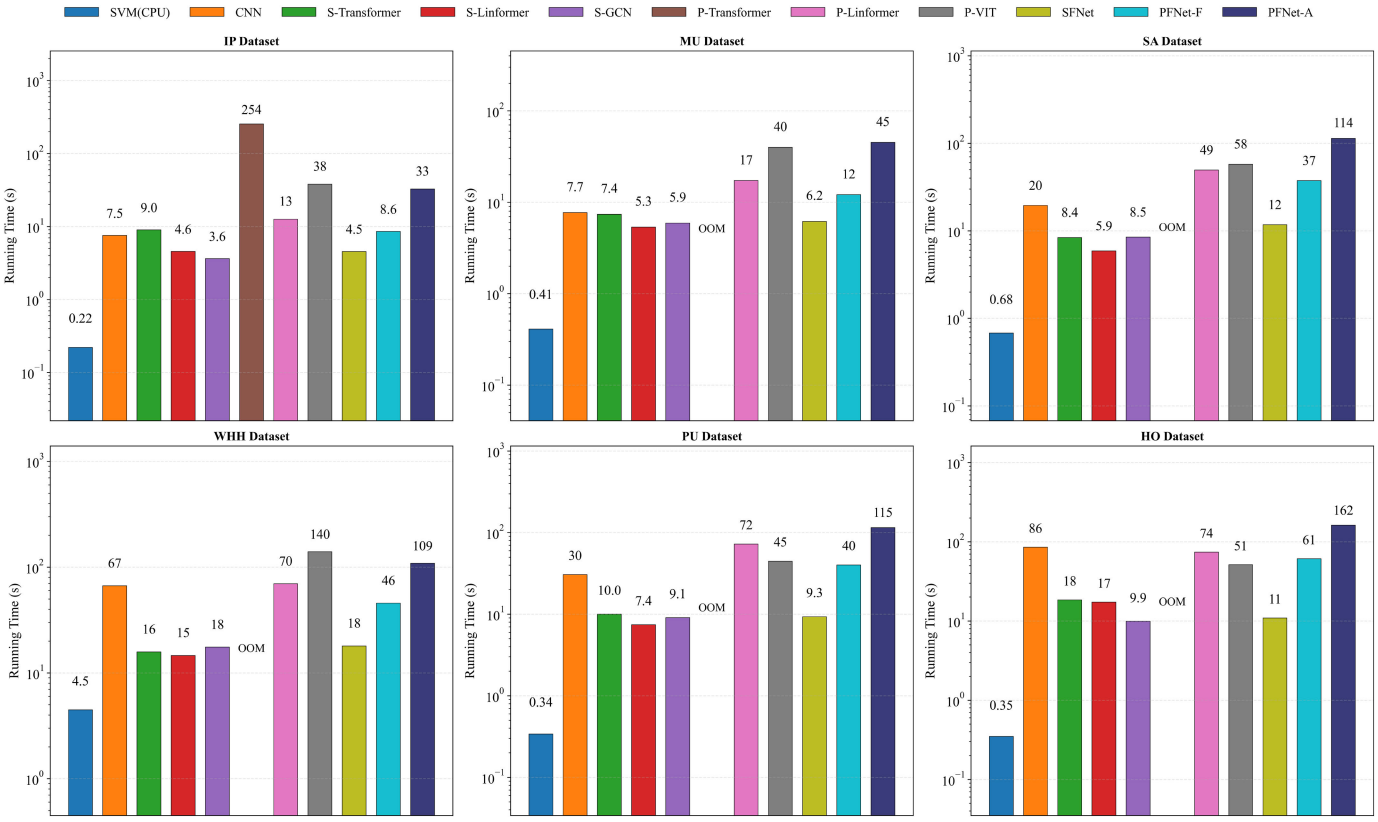


Fig. 4. Comparison of runtime across different datasets for all single-module methods (training time + testing time).

TABLE VIII

COMPARISON OF OA VALUES OBTAINED BY DIFFERENT FEATURE FUSION STRATEGIES RELATED TO (7) ACROSS SIX DATASETS. THE BEST RESULT IS SHOWN IN RED, THE SECOND-BEST IN BLUE, AND THE THIRD IN BLACK

Dataset	$\alpha = 0$ (SFNet only)	$\alpha = 0.3$ (Weighted fusion)	$\alpha = 0.5$ (Weighted fusion)	$\alpha = 0.7$ (Weighted fusion)	$\alpha = 1$ (PFNet-F only)	Gate fusion	Add (SFPNet-F)
IP	73.05 ± 3.19	79.83 ± 2.36	82.14 ± 2.31	$\mathbf{81.13 \pm 2.60}$	73.80 ± 1.29	79.01 ± 2.52	81.62 ± 2.43
MU	55.03 ± 3.52	66.65 ± 3.41	72.09 ± 2.32	72.94 ± 2.55	71.45 ± 2.97	$\mathbf{72.26 \pm 3.15}$	73.46 ± 2.56
SA	96.15 ± 1.81	99.01 ± 0.87	$\mathbf{98.13 \pm 0.94}$	97.69 ± 1.22	92.08 ± 1.50	97.94 ± 1.47	98.17 ± 1.26
WHH	84.14 ± 1.32	86.13 ± 2.73	86.84 ± 2.80	82.37 ± 1.85	75.61 ± 3.92	87.08 ± 2.30	$\mathbf{86.70 \pm 2.54}$
PU	74.15 ± 6.50	84.61 ± 4.08	88.14 ± 3.75	86.44 ± 3.86	80.13 ± 5.47	$\mathbf{86.79 \pm 3.33}$	88.36 ± 3.28
HO	63.08 ± 2.95	76.82 ± 2.86	80.08 ± 2.24	77.71 ± 2.49	75.37 ± 2.12	$\mathbf{78.73 \pm 2.55}$	81.00 ± 2.96

of the PFNet-F branch and the SFNet branch, respectively. Therefore, $\alpha = 0$ corresponds to SFNet only, while $\alpha = 1$ corresponds to PFNet-F only.

As shown in Table VIII, combining PFNet-F and SFNet generally outperforms either individual branch, indicating the strong complementarity between fine-grained propagation at the pixel-level and long-range propagation at the superpixel-level. On spatially heterogeneous datasets such as IP, MU, PU, and HO, parallel additive fusion brings clear improvements over single-branch variants. This shows that the local details captured by PFNet-F and the region-level contextual information modeled by SFNet can jointly improve classification performance in complex land-cover scenes. For datasets with relatively regular spatial distributions and more homogeneous regions, such as SA and WHH, SFNet alone already achieves strong performance. Nevertheless, parallel additive fusion further improves OA, suggesting that the local discriminative information provided by PFNet-F can effectively complement the region-level representation of SFNet.

In addition, fixed weighted fusion can achieve slightly higher OA on some datasets. For example, $\alpha = 0.5$ performs well on IP, while $\alpha = 0.3$ performs well in SA. However, the optimal weighting coefficient varies between datasets, indicating that fixed weighted fusion requires additional weight tuning at the dataset-level. Although gated fusion introduces an adaptive weight learning mechanism, it does not consistently outperform simple additive fusion on all datasets. This may be because in the small-sample semi-supervised classification setting, the additional gating parameters increase the

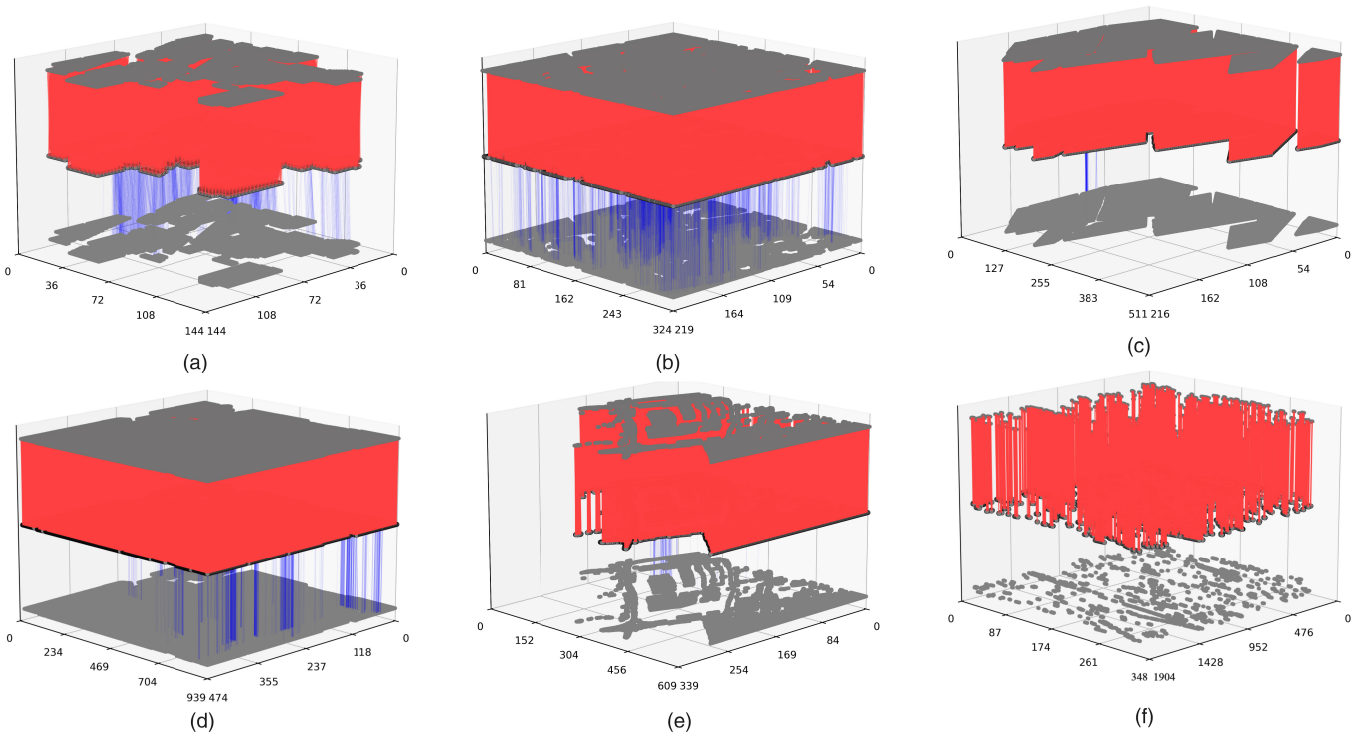


Fig. 5. 3-D visualization of sparse structural neighborhood $\mathcal{M}$ across different datasets. (a) IP. (b) MU. (c) SA. (d) WHH. (e) PU. (f) HO.

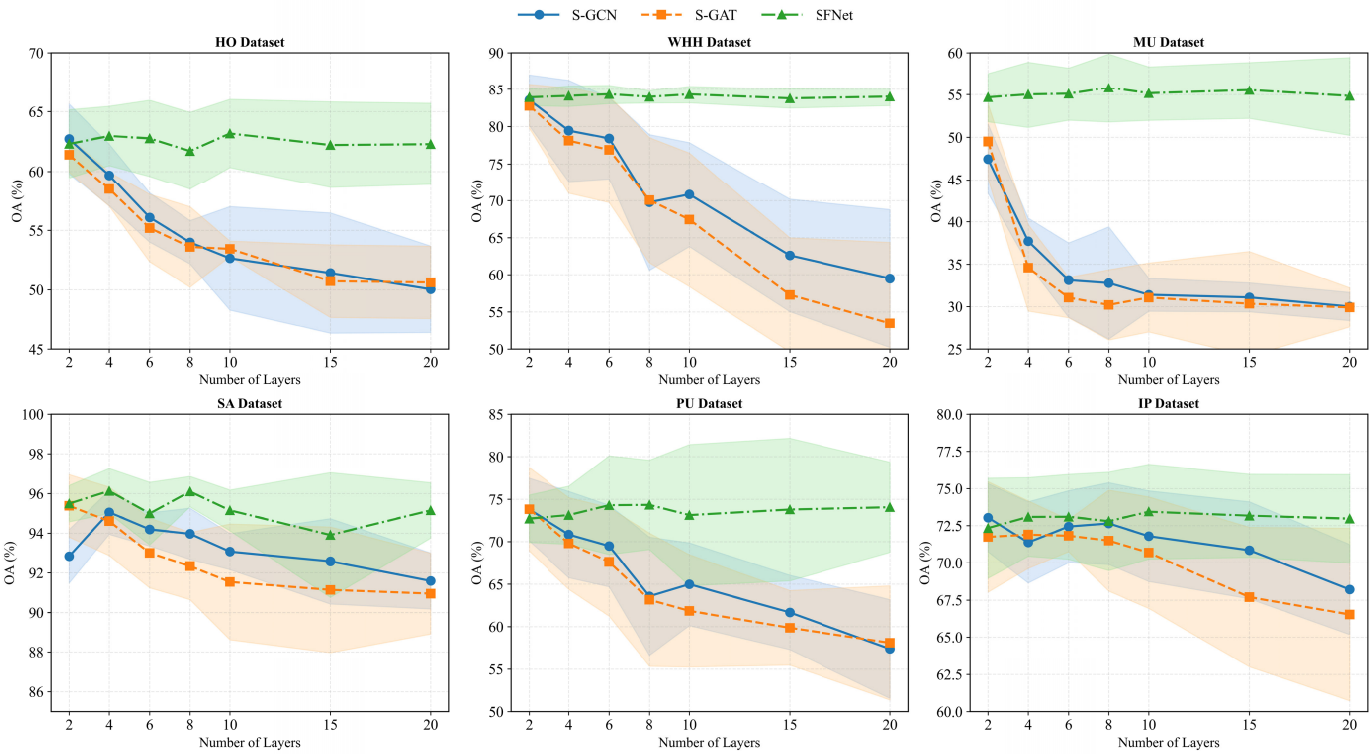


Fig. 6. Comparison of classification performance across different datasets under varying network layer conditions.

optimization difficulty and may make the model overly dependent on one branch, thereby weakening the stable complementarity between PFNet-F and SFNet. In contrast, parallel additive fusion in (7) does not introduce additional fusion hyperparameters and does not require a weight search specific to the dataset. It obtains the best OA on MU and HO and near-optimal competitive results on the remaining datasets. These results validate the

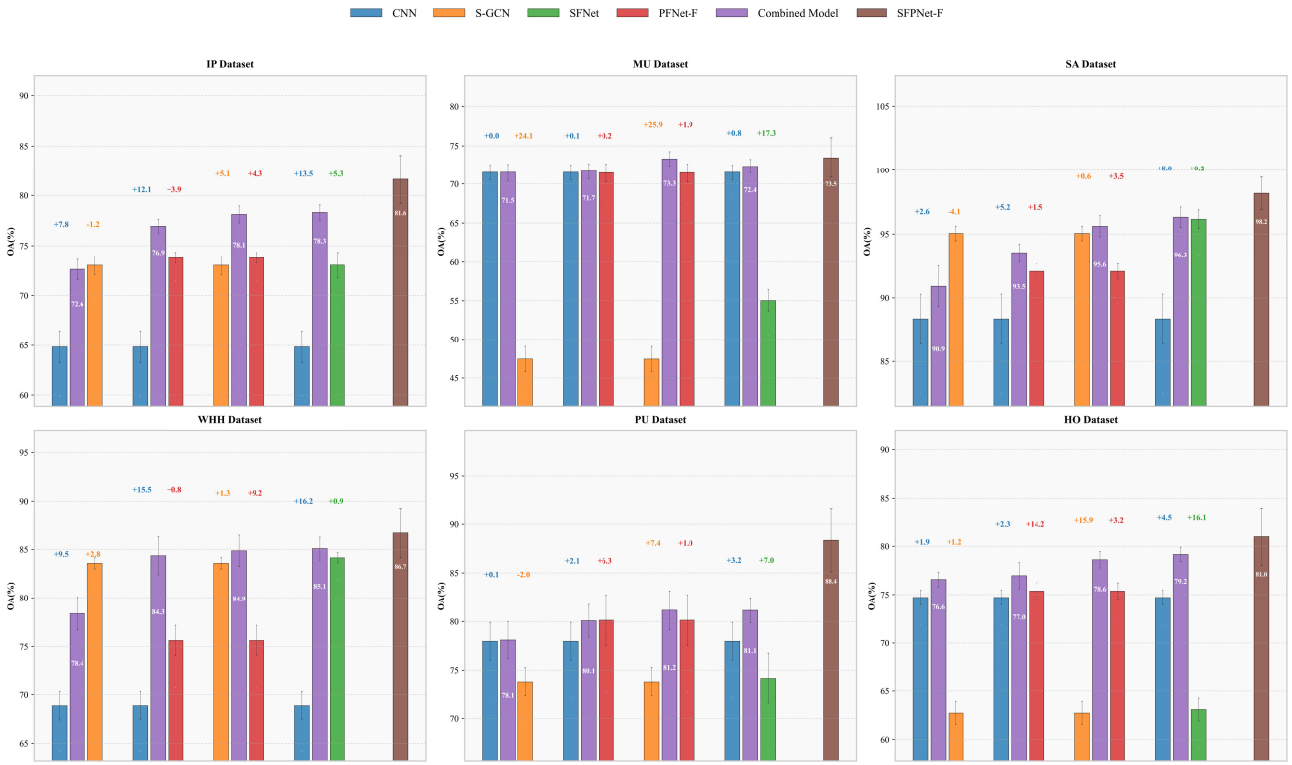


Fig. 7. Comparison of overall classification accuracy (OA) across different combination models under various datasets.

effectiveness and robustness of the parallel additive fusion mechanism.

3) Module Effectiveness Analysis: To further uncover the structural reasons behind the above performance differences, we analyze the underlying mechanisms of PFNet-F/PFNet-A and SFNet from two perspectives: 1) the quality of the constructed neighborhood structure and 2) the robustness to oversmoothing.

The effectiveness of PFNet-F/PFNet-A fundamentally relies on the high-quality sparse structured neighborhood it constructs. To examine this property, we performed a 3-D visualization on all six datasets. Specifically, we duplicated the land-cover plane into three parallel layers (top, middle, and bottom) for clarity. For each edge in the constructed neighborhood, if the two connected pixels belong to the same class, we draw a red line connecting the pixel in the middle plane to its counterpart in the top plane; otherwise, we draw a blue line connecting the middle plane to the bottom plane. As shown in Fig. 5(a)–(f), edges connecting same-class pixels dominate (red links), which ensures that feature propagation is primarily performed within semantically consistent regions and thus minimizes the influence of heterogeneous noise. Notably, on datasets with extremely dense land-cover distributions and complex class boundaries (e.g., IP and MU), we observe relatively more cross-class edges (blue links). This observation explains why PFNet-A yields more pronounced gains over PFNet-F on these datasets: in regions with more noisy connections, dynamic attention can softly suppress such heterophilic links, thereby improving the semantic purity of propagation.

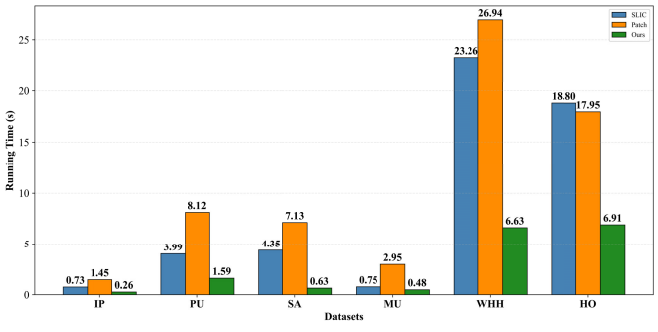


Fig. 8. Comparison of runtime across different preprocessing methods on various datasets.

We further analyze the oversmoothing robustness of SFNet from a depth-sensitivity perspective. Concretely, we compare the performance trends of GCN-S, GAT-S, and SFNet at the superpixel level under different numbers of layers (i.e., propagation steps), as reported in Fig. 6. As the depth increases, the OA of GCN-S and GAT-S exhibits a persistent degradation on most datasets, which is a typical manifestation of deep graph convolution converging toward a limiting distribution under Laplacian smoothing, where interclass distinctions are progressively weakened. For example, on MU, WHH, and PU, the OA of GCN-S and GAT-S drops by around 20 percentage points as the number of layers increases; this phenomenon is particularly evident in scenes where spectral discrepancies between classes are large. In contrast, SFNet shows substantially stronger stability within the same depth range: its OA varies only mildly across datasets, without

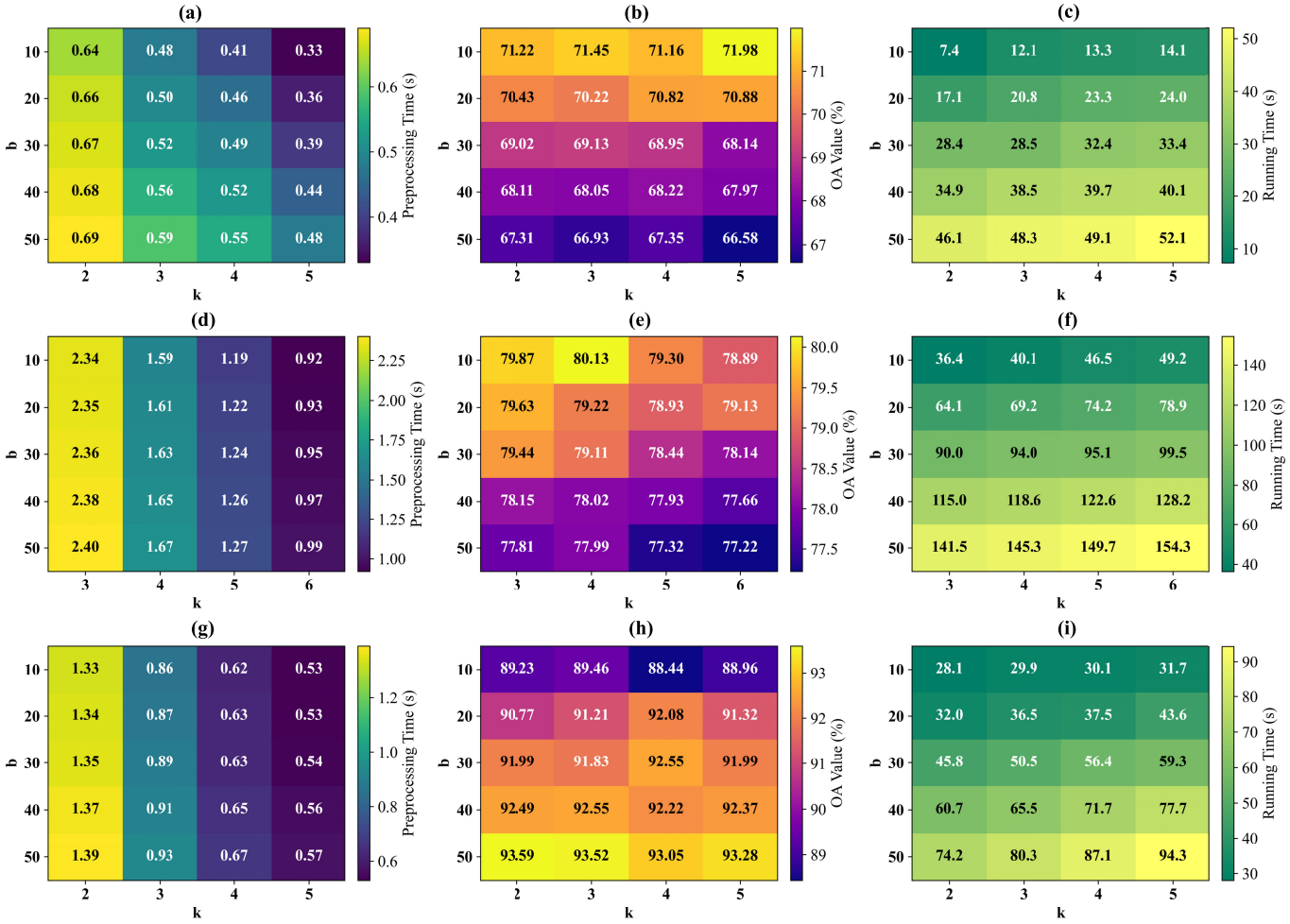


Fig. 9. Comparison of classification performance under varying hyperparameters k and b in PFNet-F across different datasets. (a) MU: preprocessing time. (b) MU: OA value. (c) MU: running time. (d) PU: preprocessing time. (e) PU: OA value. (f) PU: running time. (g) SA: Preprocessing. (h) SA: OA value. (i) SA: Running time.

systematic degradation, and in some cases even exhibits slight improvements around moderate depths. These results empirically corroborate our theoretical analysis in Theorem 2. By introducing exponential decay and injecting the initial features during propagation, SFNet induces an effective propagation kernel that moderately amplifies low-frequency components while suppressing high-frequency components in the spectral domain. Consequently, SFNet enhances large-scale structural modeling while mitigating the information dilution and oversmoothing under deep propagation, enabling robust long-range dependency modeling.

4) Potential of the Modules: After validating the standalone performance and internal mechanisms of the modules, we further investigate the potential of PFNet-F and SFNet as plug-and-play enhancement components when integrated with classical architectures. Specifically, we construct four types of composite models, namely, CNN + S-GCN, CNN + PFNet-F, S-GCN + PFNet-F, and CNN + SFNet, and compare them with the complete SFPNet-F framework. Fig. 7 reports the OA of each compound on the six datasets, together with relative improvements over its constituent baseline(s).

The results show that, after incorporating PFNet-F or SFNet, almost all composite models consistently outperform their original baselines on nearly all datasets. Taking S-GCN + PFNet-F as an example, this composite significantly outperforms CNN + S-GCN on most datasets, indicating strong complementarity between the pixel-level sparse geometric features provided by PFNet-F and the superpixel-level smoothing features provided by S-GCN. Specifically, PFNet-F strengthens local fine-grained discriminative representations, whereas S-GCN facilitates the integration of region-level consistency at the superpixel scale. Their combination therefore yields better classification performance than either component alone. Moreover, CNN + SFNet consistently achieves substantially higher OA than CNN + S-GCN across all datasets, suggesting that SFNet can provide the necessary long-range dependency modeling while preserving the inductive bias of CNNs for local textures and edges. Similarly, CNN + PFNet-F also consistently outperforms CNN + S-GCN, further highlighting the advantage of data-prior-driven sparse graph construction over regular grid convolutions and predefined superpixel graphs in representation learning. Although these composite models obtain notable gains in multiple scenarios, their overall

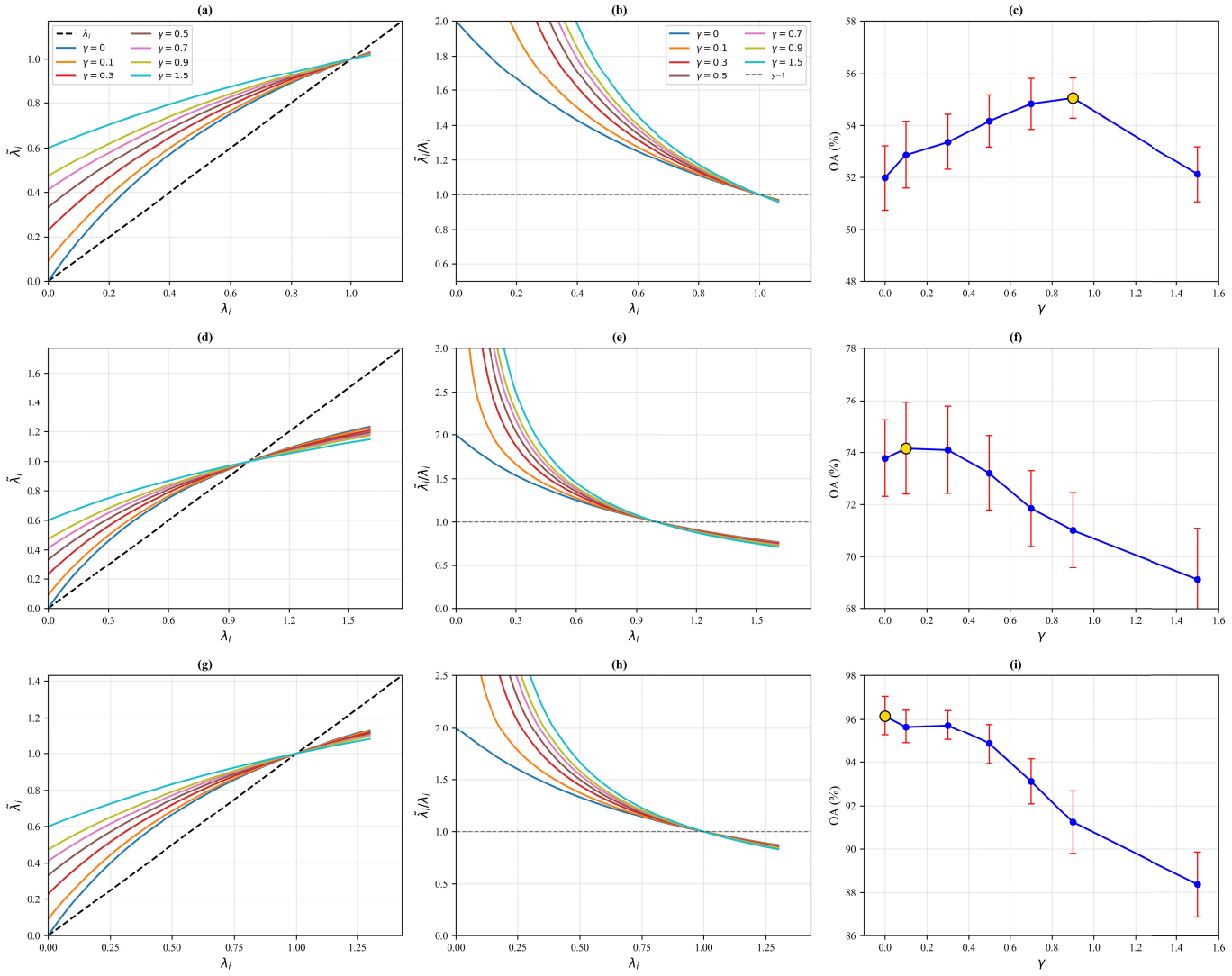


Fig. 10. Transformation curves of normalized Laplace eigenvalues $\tilde{\lambda}_i$ for feature propagation matrix $\tilde{P}$ under different γ values across various datasets, and corresponding changes in OA values. (a), (d), and (g) λ_i versus $\tilde{\lambda}_i$. (b), (e), and (h) $\tilde{\lambda}_i/\lambda_i$. (c), (f), and (i) OA versus γ.

performance still does not surpass the complete SFPNet-F framework. This observation implies that PFNet-F and SFNet are not simply additive within SFPNet-F; instead, they form a tighter synergy and complementarity under a unified sparse propagation paradigm. PFNet-F is responsible for constructing semantically fine-grained and topologically connected sparse propagation channels at the pixel scale, while SFNet robustly integrates long-range dependencies at the superpixel scale in an oversmoothing-resistant manner. Together, they shape the overall advantage of SFPNet.

In summary, through the modulewise performance comparisons, mechanism analysis, and external integration experiments, this section systematically demonstrates both the origins of SFPNet's superior performance and its extensibility. The PFNet module builds high-quality sparse neighborhoods, enabling strong fine-grained feature extraction and improved adaptability to complex scenes. The SFNet module, with theoretically supported convergence behavior and spectral filtering properties, delivers efficient and robust long-range contextual modeling. Importantly, the two modules not only serve as standalone classifiers that outperform existing methods on multiple datasets but also act as general-purpose building blocks that can be flexibly integrated into existing architectures, offering reusable and scalable structural units for future HSI analysis model design.

D. Key Hyperparameter Analysis

In this section, we conduct a systematic analysis of the key design hyperparameters in the proposed SFPNet-F/A framework, aiming to reveal their intrinsic characteristics and their impacts on both performance and efficiency. The analysis is organized into three aspects: the efficiency of the structured sparse neighborhood construction algorithm, the sensitivity of local-scale parameters in the PFNet-F branch, and the spectral behavior of the decay factor γ in SFNet together with its relationship to the precision of the classification.

First, we evaluated the computational efficiency of the proposed sparse neighborhood construction algorithm against two widely used alternatives: SLIC superpixel segmentation [42] and fixed block-based partitioning [18]. As shown in Fig. 8,

our method consistently exhibits a clear runtime advantage on all six datasets. On average, it reduces the processing time by approximately 5.90 s compared to SLIC and by approximately 8.01 s compared to the block-based strategy. These results demonstrate the practical value of the proposed construction algorithm for real-world deployment and suggest a feasible technical route for real-time HSI processing under resource-constrained settings.

Second, we investigate how the block number $b \times b$ and the neighbor number k in the construction of the sparse structured neighborhood affect both the accuracy and efficiency of the model in the pixel-level branch of PFNet-F. Fig. 9 reports, on three datasets with distinct land-cover characteristics—MU (dense and complex), PU (medium-scale and heterogeneous), and SA (regular and homogeneous)—the construction time of the sparse structured neighborhood, the OA, and the runtime of PFNet-F under different (b, k) configurations. It can be seen that the construction time increases with larger k and decreases with smaller b; nevertheless, the overall cost remains at a relatively low level. Regarding OA, on the SA dataset with regular structures and strong homogeneity, performance is relatively insensitive to variations in k and b, indicating that such simple scenes have weaker dependence on fine-grained contextual modeling. In contrast, on MU and PU, which exhibit substantial heterogeneity, PFNet-F is more sensitive to k: increasing k may introduce redundant or noisy information and thus degrade performance. Meanwhile, the runtime of PFNet-F increases with larger k and larger b, yet strong performance can still be achieved with relatively small k, further supporting the rationale for using PFNet-F to capture fine-grained dependencies efficiently.

Finally, we analyze the spectral filtering behavior of the decay factor γ in SFNet and its impact on classification performance. Fig. 10(a)–(c), (d)–(f), and (g)–(i) presents, for MU, PU, and SA respectively, the eigenvalue transformation curves of the normalized Laplacian associated with the feature propagation matrix $\tilde{\mathbf{P}}$ and the corresponding OA values under different γ. As γ increases, the eigenvalue ratio $\tilde{\lambda}_i/\lambda_i$ is noticeably amplified in the low-frequency regime while being further suppressed in the high-frequency regime. This spectral observation confirms that a larger γ strengthens the low-pass filtering effect, making the model more inclined to aggregate smooth global structural information while more aggressively attenuating high-frequency details and noise. Consistent with land-cover characteristics, on MU with strong heterogeneity and pronounced noise, a larger γ enhances low-pass filtering and suppresses high-frequency perturbations, leading to a stable improvement in classification accuracy. In SA with clean and well-structured regions, an excessively large γ can oversmooth the necessary details and thus damage the performance, making smaller γ values more suitable. PU shows a preference for moderate γ, indicating the need for a balanced tradeoff between noise smoothing and detail preservation. Overall, these results suggest that γ acts as a "smoothing regulator" that adapts to different land-cover distributions: larger values are preferable in complex scenes to suppress noise, whereas smaller values are preferable in homogeneous scenes to retain discriminative details.

V. Conclusion

This work systematically investigates the intrinsic tension between expressive power and computational efficiency in feature propagation models for HSI classification. Conventional pixel-level methods can capture fine-grained representations; yet, they are often constrained by either global quadratic computation or limited local receptive fields. In contrast, superpixel-level methods improve efficiency but may sacrifice pixel-level details due to the region homogeneity assumption. To jointly address these limitations, we propose a sparse feature propagation framework, termed SFPNet-F/A. The framework consists of two tightly coupled core modules. The first is a pixel-level sparse feature propagation module, PFNet-F/PFNet-A, which constructs a high-quality structured sparse neighborhood via a joint spectral–spatial similarity metric, and performs pixelwise propagation with either fixed or dynamic weights, thereby focusing on fine-grained dependency modeling among semantically similar pixels. The second is a superpixel-level sparse feature propagation module, SFNet, which introduces a multihop propagation mechanism with decay aggregation and state preservation on the superpixel graph. We further provide a spectral analysis that establishes its convergence and low-pass filtering characteristics, effectively mitigating the oversmoothing issue that commonly arises in deep GNNs. Through the sparse collaboration of dual pathways at the pixel-level and superpixel-level, SFPNet-F/A unifies fine-grained representation learning and long-range context modeling within a single-computational paradigm.

Extensive experiments on six public HSI benchmarks demonstrate the effectiveness and robustness of the proposed method. Under extremely limited-label settings, both SFPNet variants consistently outperform strong GNN- and Transformer-based baselines in terms of OA, AA, and the Kappa coefficient. In addition, the evaluations stratified by land-cover scale indicate that the two variants of SFPNet remain leading on both complex small-scale patches and homogeneous large-scale regions, confirming their cross-scale consistent modeling capability. Module-level ablations and visualization analyses further show that the structured sparse neighborhood constructed by PFNet-F/PFNet-A achieves a favorable balance between intraclass connectivity and interclass noise suppression, while SFNet efficiently captures long-range dependencies through its feature propagation operator. In particular, the two modules can function not only as standalone high-performance classifiers but also as plug-and-play enhancement components that can be flexibly integrated into existing architectures, providing reusable building blocks for future HSI feature propagation model design.

Despite the competitive results achieved in this study, several directions merit further exploration. First, with respect to geometric prior construction, the current structured neighborhood and superpixel topology still rely on predefined segmentation and fixed hyperparameters. Future work may consider learnable neighborhood construction strategies and multiscale adaptive superpixel graphs and further extend the formulation to hypergraphs or spatiotemporal graphs to better model complex scenes and multitemporal data. Second, in terms of learning paradigms and task extensions, it would be

promising to combine sparse feature propagation mechanisms with self-supervised pretraining to better exploit large-scale unlabeled HSIs and to investigate transferability and generalization to downstream tasks such as change detection, object detection, unmixing, and inversion, ultimately moving toward a unified and scalable sparse representation learning framework for hyperspectral analysis.

Appendix A
Relevant Theories And Their Proofs

Lemma 1: Let the input data cube on a regular grid be $X \in \mathbb{R}^{H\times W\times C}$, where $H, W \in \mathbb{Z}^+$ denote the height and width of the grid, and $HW \geq 5$. Each pixel i corresponds to a feature vector $X_i \in \mathbb{R}^C$, $i \in \{0,\ldots,N-1\}$, and a grid coordinate $C_i = (\lfloor i/W \rfloor,\ i \bmod W)$. Consider the neighborhood of pixel i defined as follows:

$$\mathcal{N}(i) = \left\{ j \;\middle|\; j \in \text{Top-}k\Big(\exp\big(-\left\|C_i - C_q\right\|_2^2/(2\gamma)\big) \quad q \in \{0,\ldots,N-1\}\Big)\right\} \tag{B1}$$

where $\gamma > 0$, and Top-$k(\cdot)$ select the k pixels with the largest Gaussian-kernel similarity of all pixels. When the neighbor number satisfies $HW \geq k \geq 5$ with $k \in \mathbb{Z}^+$, the graph G induced by the neighborhood set $\mathcal{N} = \{\mathcal{N}(i)\}_{i=0}^{N-1}$ is connected over all pixels.

Proof: Fix a pixel i and its grid coordinate C_i. Since the Gaussian kernel $\exp(-\|C_i - C_j\|^2/(2\gamma))$ is monotonically decreasing with respect to the squared Euclidean distance $\|C_i - C_j\|^2$, the neighborhood $\mathcal{N}(i)$ consists of the k pixels that are closest to the pixel i in terms of the Euclidean distance of the grid. When $k > 5$, the edge set only increases and the connectivity does not change. Therefore, it is sufficient to prove that G is connected when $k = 5$. We discuss three cases according to the grid size.

Case 1: $H \geq 3$ and $W \geq 3$. Consider an arbitrary 3×3 grid. For the center pixel i, the five closest points in the Euclidean distance are exactly the pixel itself and its four direct neighbors in the up, down, left, and right directions. Therefore, when $k = 5$

$$\mathcal{N}(i) = \Big\{i,\ i-1,\ i+1,\ i-W,\ i+W\Big\}. \tag{B2}$$

It follows that $\mathcal{N}(i) \cap \{\mathcal{N}(j) : j \in \{0,\ldots,8\}\} \neq \emptyset$. By the definition of graph connectivity, if the neighborhoods of two vertices have a nonempty intersection, then they belong to the same connected component. Hence, this 3×3 region forms a connected component. Any $H \times W$ grid can be viewed as covered by overlapping 3×3 regions that expand outward from the interior. Therefore, the entire graph G is connected.

Case 2: $H = 1$ and $W \geq 5$, or symmetrically $W = 1$ and $H \geq 5$. In this case, the grid degenerates into a line. For any interior pixel i on the line, the closest points in Euclidean distance are exactly the pixel itself and its left and right neighbors. Therefore, when $k = 3$

$$\mathcal{N}(i) = \Big\{i,\ i-1,\ i+1\Big\}. \tag{B3}$$

Hence, all pixels on the line are connected sequentially through adjacency relations, and the graph is connected. When k increases to 5, connectivity remains unchanged.

Case 3: $H = 2$ and $W \geq 3$, or symmetrically $W = 2$ and $H \geq 3$. Consider a 2×3 grid. For a noncorner pixel i in the first row, the four closest points in Euclidean distance are the pixel itself and its left, right, and down neighbors. Therefore, when $k = 4$

$$\mathcal{N}(i) = \Big\{i,\ i-1,\ i+1,\ i+W\Big\}. \tag{B4}$$

It follows that $\mathcal{N}(i) \cap \{\mathcal{N}(j) : j \in \{0,\ldots,5\}\} \neq \emptyset$. Hence, this 2×3 region forms a connected component. Any $2 \times W$ grid or $H \times 2$ grid can be viewed as covered by overlapping 2×3 regions (or 3×2 regions) from the interior to the exterior. Therefore, the entire graph G is connected. When k increases to 5, connectivity is still preserved.

In conclusion, for any input grid with $HW \geq 5$, when $HW \geq k \geq 5$ the graph G defined by the neighborhood set $\mathcal{N}$ is connected. □

Theorem 3: Let the input data cube on a regular grid be $X \in \mathbb{R}^{H\times W\times C}$, where $H, W \in \mathbb{Z}^+$ denote the grid height and width, and $HW \geq 5$. Each pixel i corresponds to a feature vector $X_i \in \mathbb{R}^C$, $i \in \{0,\ldots,N-1\}$, and a grid coordinate $C_i = (\lfloor i/W \rfloor,\ i \bmod W)$. Let the structured neighborhood set $\mathcal{M} = \{\mathcal{M}(i)\}_{i=0}^{N-1}$ be constructed by Algorithm 1. If the neighbor number k satisfies $HW/B^2 \geq k \geq 5$, with $k, B \in \mathbb{Z}^+$, then the graph G defined by the structured neighborhood $\mathcal{M}$ is connected.

Proof: Steps 1 and 2 of Algorithm 1 form a double loop. This procedure can be viewed as traversing the input X using a sliding window of size $(H/B) \times (W/B)$ with steps $H/(2B)$ and $W/(2B)$ along the two spatial directions. This traversal partitions X into $(2B-1)^2$ subgrids, each of size $(H/B) \times (W/B)$.

We first consider a simplified setting where the strides are H/B and W/B. In this case, the sliding window partitions X into B^2 nonoverlapping subgrids of size $(H/B) \times (W/B)$. According to Step 3 of Algorithm 1, within each subgrid we first perform the local neighborhood selection

$$\mathcal{M}(i) = \left\{ j \;\middle|\; j \in \text{Top-}5\Big(\exp\big(-\|C_i - C_q\|^2/2\big) \quad q \in \{0,\ldots,N-1\}\Big)\right\}. \tag{B5}$$

By Lemma 1, when each subgrid satisfies $HW/B^2 \geq k \geq 5$ and contains at least five pixels, the graph g_m induced by this neighborhood within each subgrid, where $m \in \{0,\ldots,B^2\}$ is connected. Furthermore, in step 4 Algorithm 1 adds up to $k-5$ additional neighbors for each $\mathcal{M}(i)$ through an extra metric function and a Top-$(k-5)$ operation. Adding edges to a connected graph does not destroy its connectivity. Therefore, the induced graph g_m within each subgrid remains connected when $k \geq 5$.

We now consider the actual steps $H/(2B)$ and $W/(2B)$. In this case, adjacent sliding windows overlap 50% along both the row and column directions. This means that each subgrid that did not overlap under steps H/B and W/B now shares a subset of pixels with its neighboring subgrids in the up, down, left, and right directions. Since Algorithm 1 repeats the same neighborhood construction procedure within every

subgrid, under the actual strides $H/(2B)$ and $W/(2B)$, the graph g_p induced within each subgrid, where $p \in \{0, \ldots, (2B-1)^2\}$, is connected.

Finally, let g_p and g_q be two connected graphs corresponding to two spatially adjacent subgrids. Due to window overlap, their node sets have a nonempty intersection. Therefore, $g_p \cup g_q$ forms a larger connected graph. By repeatedly merging all adjacent subgrids that overlap, the global graph $G = \bigcup_{p=0}^{(2B-1)^2} g_p$ is connected. In summary, for any input cube with $HW \geq 5$, under the condition $HW/B^2 \geq k \geq 5$, the graph G defined by the structured neighborhood $\mathcal{M}$ is connected. □

Theorem 4: Let the normalized propagation matrix with self-loops be $P = \tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$, whose associated Laplacian has eigenvalues $\lambda_i \in [0, 2)$. Let the SFNet limiting propagation matrix be $\tilde{P} = \sum_{i=1}^{\infty}(\beta P)^i$, where $\beta = (2+\gamma)^{-1}$ and $\gamma \in \mathbb{R}^+$. Denote the eigenvalues of the Laplacian associated with $\tilde{P}$ as $\tilde{\lambda}_i$. Then λ_i and $\tilde{\lambda}_i$ share the same set of eigenvectors. Moreover, when $\lambda_i \in [0, 1]$, it holds that $\lambda_i \leq \tilde{\lambda}_i$. When $\lambda_i \in (1, 2)$, it holds that $\lambda_i \geq \tilde{\lambda}_i$.

Proof: Let the Laplacian associated with P be $L = I - P$, with eigenvalues λ_i. Let the Laplacian associated with $\tilde{P}$ be $\tilde{L} = I - \tilde{P}$, with eigenvalues $\tilde{\lambda}_i$. Since L is symmetric and positive semidefinite, it admits the spectral decomposition as follows:

$$\begin{aligned} L &= U\Lambda U^{\mathsf{T}} \\ P &= U\,(I-\Lambda)\,U^{\mathsf{T}} \end{aligned} \tag{B6}$$

where $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_{|S|})$ is the eigenvalue matrix and $U = [u_1, \ldots, u_{|S|}]$ denotes the orthogonal eigenvector matrix. For any nonnegative integer $r \geq 0$, we obtain $(\beta P)^r = U[\beta(I - \Lambda)]^r U^{\mathsf{T}}$. Given the spectral radius $\rho(\beta P) < 1$, the convergence condition of the geometric series ensures that

$$\begin{aligned} \tilde{P} &= \sum_{i=1}^{\infty}(\beta P)^i \\ &= U\left[\sum_{i=1}^{\infty}(\beta\,(I-\Lambda))^i\right]U^{\mathsf{T}} \\ &= U\left[(I-\beta\,(I-\Lambda))^{-1} - I\right]U^{\mathsf{T}}. \end{aligned} \tag{B7}$$

Therefore, $\tilde{P}$ can be represented as a diagonal matrix on the same orthogonal basis U. This implies that $\tilde{P}$ and P share the same set of eigenvectors, and consequently λ_i and $\tilde{\lambda}_i$ share the same eigenvectors.

Let the eigenvalues of P be $\mu_i = 1 - \lambda_i$, where $\mu_i \in (-1, 1]$. Substituting $\beta = (2+\gamma)^{-1}$, the eigenvalues of $\tilde{P}$ can be written as follows:

$$\tilde{\mu}_i = \frac{1}{1-\mu_i\,(2+\gamma)^{-1}} - 1 = \frac{\mu_i}{2+\gamma-\mu_i}. \tag{B8}$$

Hence, the eigenvalues of $\tilde{L}$ are

$$\tilde{\lambda}_i = 1 - \frac{1-\lambda_i}{2+\gamma-(1-\lambda_i)} = \frac{\gamma+2\lambda_i}{1+\gamma+\lambda_i}. \tag{B9}$$

Define the difference function between $\tilde{\lambda}_i$ and λ_i as follows:

$$F(\lambda_i) = \frac{\gamma+2\lambda_i}{1+\gamma+\lambda_i} - \lambda_i = \frac{(1-\lambda_i)\,(\gamma+\lambda_i)}{1+\gamma+\lambda_i}. \tag{B10}$$

Since the denominator $1 + \gamma + \lambda_i > 0$ because $\gamma > 0$ and $\lambda_i \geq 0$ and $\gamma + \lambda_i > 0$, the sign of $F(\lambda_i)$ is determined solely by the factor $(1 - \lambda_i)$. Therefore, when $\lambda_i \in [0, 1]$, we have $F(\lambda_i) \geq 0$ and thus $\lambda_i \leq \tilde{\lambda}_i$. When $\lambda_i \in (1, 2)$, we have $F(\lambda_i) < 0$ and thus $\lambda_i \geq \tilde{\lambda}_i$. □

REFERENCES

[1] M. D. F. O. Baffa, D. M. Zezell, L. Bachmann, T. M. Pereira, T. M. Deserno, and J. C. Felipe, "Deep neural networks can differentiate thyroid pathologies on infrared hyperspectral images," *Comput. Methods Programs Biomed.*, vol. 247, Apr. 2024, Art. no. 108100.

[2] C. Lou et al., "Land use/land cover (LULC) classification using hyperspectral images: A review," *Geo-spatial Inf. Sci.*, vol. 28, no. 2, pp. 345–386, Mar. 2025.

[3] D. Hong et al., "Hyperspectral imaging," 2025, *arXiv:2508.08107*.

[4] J. Zhao et al., "A camouflage target classification method based on spectral difference enhancement and pixel-pair features in land-based hyperspectral images," *Eng. Appl. Artif. Intell.*, vol. 156, Sep. 2025, Art. no. 111141.

[5] J. Li, J. M. Bioucas-Dias, and A. Plaza, "Semisupervised hyperspectral image segmentation using multinomial logistic regression with active learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 48, no. 11, pp. 4085–4098, Nov. 2010.

[6] G. Mercier and M. Lennon, "Support vector machines for hyperspectral image classification with spectral-based kernels," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, vol. 1, Jul. 2003, pp. 288–290.

[7] Y. Zhang, G. Cao, X. Li, and B. Wang, "Cascaded random forest for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 4, pp. 1082–1094, Apr. 2018.

[8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[9] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 7132–7141.

[10] Iqra and K. J. Giri, "SO-YOLOv8: A novel deep learning-based approach for small object detection with YOLO beyond COCO," *Expert Syst. Appl.*, vol. 280, Jun. 2025, Art. no. 127447.

[11] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Oct. 2016.

[12] J. Xiao, J. Li, Q. Yuan, and L. Zhang, "A dual-UNet with multistage details injection for hyperspectral image fusion," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–13, 2021.

[13] M. Q. Alkhatib et al., "Tri-CNN: A three branch model for hyperspectral image classification," *Remote Sens.*, vol. 15, no. 2, p. 316, Jan. 2023.

[14] Y. Zhou, X. Huang, X. Yang, J. Peng, and Y. Ban, "DCTN: Dual-branch convolutional transformer network with efficient interactive self-attention for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5508616.

[15] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Represent.*, 2017, pp. 1–14.

[16] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–12.

[17] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1024–1034.

[18] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5998–6008.

[19] C. K. Joshi, "Transformers are graph neural networks," 2025, *arXiv:2506.22084*.

[20] A. Qin, Z. Shang, J. Tian, Y. Wang, T. Zhang, and Y. Y. Tang, "Spectral–spatial graph convolutional networks for semisupervised hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 2, pp. 241–245, Feb. 2019.

[21] A. Yang, M. Li, Y. Ding, D. Hong, Y. Lv, and Y. He, "GTFN: GCN and transformer fusion network with spatial–spectral features for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 6600115.

[22] N. Guo et al., "PGNN-net: Parallel graph neural networks for hyperspectral image classification using multiple spatial–spectral features," *Remote Sens.*, vol. 16, no. 18, p. 3531, Sep. 2024.

[23] A. Yang, M. Li, Y. Ding, X. Xiao, and Y. He, "An efficient and lightweight spectral–spatial feature graph contrastive learning framework for hyperspectral image clustering," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5537714.

[24] S. Huang et al., "S2GFormer: A transformer and graph convolution combining framework for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5537514.

[25] D. Hong et al., "SpectralFormer: Rethinking hyperspectral image classification with transformers," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–15, 2021.

[26] Y. Feng, J. Zhu, R. Song, and X. Wang, "S2EFT: Spectral–spatial-elevation fusion transformer for hyperspectral image and LiDAR classification," *Knowl.-Based Syst.*, vol. 283, Jan. 2024, Art. no. 111190.

[27] N. Chen, L. Fang, Y. Xia, S. Xia, H. Liu, and J. Yue, "Spectral query spatial: Revisiting the role of center pixel in transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5402714.

[28] L. Sun, H. Zhang, Y. Zheng, Z. Wu, Z. Ye, and H. Zhao, "MASSFormer: Memory-augmented spectral–spatial transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5516415.

[29] S. Jia, Y. Wang, S. Jiang, and R. He, "A center-masked transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5510416.

[30] Y. Zhou, X. Huang, X. Yang, J. Peng, Y. Ban, and N. Jiang, "MSMT-LCL: Multiscale spatial–spectral masked transformer with local contrastive learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5534316.

[31] T. Wang, Y. Sun, and Y. Hu, "Multiscale graph transformer network with dynamic superpixel pyramid for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 18, pp. 13438–13456, 2025.

[32] A. Yang, M. Li, Y. Ding, Y. He, M. Bi, and Q. Zheng, "CTFN: Multiscale CNN and transformer with graph encodings fusion network for hyperspectral image classification," *Expert Syst. Appl.*, vol. 288, Sep. 2025, Art. no. 128063.

[33] S. Wan, C. Gong, P. Zhong, B. Du, L. Zhang, and J. Yang, "Multiscale dynamic graph convolutional network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 5, pp. 3162–3177, May 2020.

[34] Q. Liu, L. Xiao, J. Yang, and Z. Wei, "CNN-enhanced graph convolutional network with pixel- and superpixel-level feature fusion for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 10, pp. 8657–8671, Oct. 2021.

[35] Q. Liu, L. Xiao, J. Yang, and Z. Wei, "Multilevel superpixel structured graph U-nets for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–15, 2021.

[36] Y. Dong, Q. Liu, B. Du, and L. Zhang, "Weighted feature fusion of convolutional neural network and graph attention network for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 31, pp. 1559–1572, 2022.

[37] H. Zhou, F. Luo, H. Zhuang, Z. Weng, X. Gong, and Z. Lin, "Attention multihop graph and multiscale convolutional fusion network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5508614.

[38] Y. Lu, S. Mei, F. Xu, M. Ma, and X. Wang, "Separable deep graph convolutional network integrated with CNN and prototype learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5516216.

[39] W. Liu, Y. Zhang, and Y. Dong, "Multifeature collaborative attention dynamic hypergraph convolutional network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025, Art. no. 5522115.

[40] X. Zhao et al., "GTCFN: A graph-based transformer and convolution fusion network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025, Art. no. 5530819.

[41] X. Liu, A. H.-M. Ng, L. Ge, F. Lei, and X. Liao, "Multibranch fusion: A multibranch attention framework by combining graph convolutional network and CNN for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5528817.

[42] R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua, and S. Süsstrunk, "SLIC superpixels compared to state-of-the-art superpixel methods," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 34, no. 11, pp. 2274–2282, Nov. 2012.

[43] J. Zhu, Y. Yan, L. Zhao, M. Heimann, L. Akoglu, and D. Koutra, "Beyond homophily in graph neural networks: Current limitations and effective designs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 7793–7804.

[44] H. Dai, Z. Kozareva, B. Dai, A. Smola, and L. Song, "Learning steady-states of iterative algorithms over graphs," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1106–1114.

[45] K. Xu, C. Li, Y. Tian, T. Sonobe, K.-I. Kawarabayashi, and S. Jegelka, "Representation learning on graphs with jumping knowledge networks," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5453–5462.

[46] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional block attention module," in *Proc. Eur. Conf. Comput. Vis.*, Sep. 2018, pp. 3–19.

[47] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. 18th Int. Conf. Med. Image Comput. Comput.-Assist. Intervent.*, 2015, pp. 234–241.

[48] X. Zhao, J. Ma, L. Wang, Z. Zhang, Y. Ding, and X. Xiao, "A review of hyperspectral image classification based on graph neural networks," *Artif. Intell. Rev.*, vol. 58, no. 6, p. 172, Mar. 2025.

[49] C. Li, B. Zhang, D. Hong, X. Jia, A. Plaza, and J. Chanussot, "Learning disentangled priors for hyperspectral anomaly detection: A coupling model-driven and data-driven paradigm," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 4, pp. 6883–6896, Apr. 2025.

[50] G. Zhang, S. Zhang, and G. Yuan, "Bayesian graph local extrema convolution with long-tail strategy for misinformation detection," *ACM Trans. Knowl. Discovery Data*, vol. 18, no. 4, pp. 1–21, May 2024.

[51] Y. Lin, M. Yang, J. Yu, P. Hu, C. Zhang, and X. Peng, "Graph matching with bi-level noisy correspondence," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 23305–23314.

[52] Q. Wu, W. Zhao, Z. Li, D. P. Wipf, and J. Yan, "NodeFormer: A scalable graph structure learning transformer for node classification," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 27387–27401.

[53] D. Han, X. Pan, Y. Han, S. Song, and G. Huang, "FLatten transformer: Vision transformer using focused linear attention," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 5938–5948.

[54] C. Deng, Z. Yue, and Z. Zhang, "Polynormer: Polynomial-expressive graph transformer in linear time," 2024, *arXiv:2403.01232*.

[55] Z. Qin et al., "CosFormer: Rethinking softmax in attention," 2022, *arXiv:2202.08791*.

[56] Y. Luo, L. Shi, and X.-M. Wu, "Classic GNNs are strong baselines: Reassessing GNNs for node classification," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 97650–97669.

[57] X. Li, Y. Zhang, Y. Xu, and X. Xu, "NWGformer: A linear graph transformer with non-linear re-weighting of attention scores," *Knowl.-Based Syst.*, vol. 332, Jan. 2026, Art. no. 114815.

[58] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, "Simplifying graph convolutional networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6861–6871.

[59] A. Ng, M. Jordan, and Y. Weiss, "On spectral clustering: Analysis and an algorithm," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 14, 2001, pp. 849–856.

[60] A. Paszke et al., "Automatic differentiation in pytorch," in *Proc. NeurIPS Workshop Autodiff*, 2017, pp. 1–4.